Software Engineering Design Capstone Project Final Documentation

UniExpert: AI-Powered Expert Discovery & Academic Collaboration Platform

---

Semester: Fall 2026

Team Members:

1. **Abu Zafor** ID: 232-35-331
2. **Anamul Haque** ID: 232-35-285

Batch: 41 **Section:** A

Course Code: SE331

Course Name: Software Engineering Design Capstone Project

Course Teacher Name: Md. Shohel Arman

Designation: Assistant Professor, Department of SWE

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**
1.1 Objective 2
1.2 Key Features 2

CHAPTER 2: SYSTEM ANALYSIS **4**
2.1 Scenario Writing 4
2.1.1 Scenario 1: Formulating an Unstructured Capstone Problem and Extracting Skill Entities 4
2.1.2 Scenario 2: Semantic Hybrid Expert Discovery and Reviewing XAI Explanations 4
2.1.3 Scenario 3: Detecting Multidisciplinary Skill Gaps and Recruiting Co-Advisors 5
2.1.4 Scenario 4: Negotiating Collaboration Requests via Finite State Machine 5
2.1.5 Scenario 5: Managing Faculty Quota Exhaustion and Scheduling Office Hours 5
2.1.6 Scenario 6: Verifying Academic Credentials and Generating Compliance Audit Reports 6
2.2 Stakeholders 6
2.3 User Profiles 7
2.3.1 User Profile-01: Student Project Lead 8
2.3.2 User Profile-02: Faculty Advisor / Academic Expert 9
2.3.3 User Profile-03: Industry Mentor 10
2.3.4 User Profile-04: Department Capstone Coordinator 11
2.3.5 User Profile-05: System Administrator 12
2.4 Scope 12
2.4.1 Scope Boundaries 14
2.5 Feasibility Study 14
2.6 Project Block Diagram 16
2.6.1 Block Diagram-1: Student Project Lead 16
2.6.2 Block Diagram-2: Faculty Advisor 16
2.6.3 Block Diagram-3: Industry Mentor 17
2.6.4 Block Diagram-4: Department Capstone Coordinator 17
2.6.5 Block Diagram-5: System Administrator 17

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION **18**
3.1 Functional Requirements 18
3.2 Non-Functional Requirements 21
3.2.1 Security 21
3.2.2 Privacy 22
3.2.3 Reliability 22
3.2.4 Performance 23
3.2.5 Accuracy 23

3.2.6 Usability . . . 23
3.2.7 Operational . . . 24

CHAPTER 4: USE CASE MODELLING . . . 25
4.1 Use Case Diagram . . . 25
4.2 Use Case Description . . . 26
4.2.1 Use Case Description-01: Register Account & Verify Academic Domain . 27
4.2.2 Use Case Description-02: Log In & Session Authentication . . . 28
4.2.3 Use Case Description-03: Manage Student Research Profile & Portfolio . 29
4.2.4 Use Case Description-04: Ingest and Manage Faculty Expert Profile . . . 30
4.2.5 Use Case Description-05: Verify Faculty Academic Credentials & Tier Assignment . . . 31
4.2.6 Use Case Description-06: Ingest Problem Statement & Natural Language Parsing . . . 32
4.2.7 Use Case Description-07: Extract Domain Taxonomy & Prerequisite Skill Entities . . . 33
4.2.8 Use Case Description-08: Refine Extracted Requirements & Skill Weights 34
4.2.9 Use Case Description-09: Execute Dense Semantic Vector Search . . . 35
4.2.10 Use Case Description-10: Multi-Criteria Hybrid Match Ranking & Scoring 36
4.2.11 Use Case Description-11: Generate Explainable AI (XAI) Recommendation Breakdown . . . 37
4.2.12 Use Case Description-12: Filter Candidates by Department, Quota & Availability . . . 38
4.2.13 Use Case Description-13: Perform Side-by-Side Expert Comparison . . . 39
4.2.14 Use Case Description-14: Analyze Multidisciplinary Team Skill Gap ($\Delta\mathcal{K}$) 40
4.2.15 Use Case Description-15: Run Set-Cover Optimization for Co-Advisor Selection . . . 41
4.2.16 Use Case Description-16: Submit Collaboration Advisory Request . . . 42
4.2.17 Use Case Description-17: Manage Advisory Request Lifecycle (Accept / Decline / Negotiate) . . . 43
4.2.18 Use Case Description-18: Enforce Faculty Advisory Quotas & Blackout Windows . . . 44
4.2.19 Use Case Description-19: Schedule Mentorship Consultation Session . . . 45
4.2.20 Use Case Description-20: Exchange Contextual Messages & Project Milestones . . . 46
4.2.21 Use Case Description-21: Submit Post-Collaboration Evaluation & Endorsement . . . 47
4.2.22 Use Case Description-22: Moderate Content & Re-Index Ontology Taxonomy 48
4.2.23 Use Case Description-23: Monitor Departmental Supervisory Allocation Dashboard . . . 49
4.2.24 Use Case Description-24: Search and Export Institutional Compliance Audit Trail . . . 50
4.2.25 Use Case Description-25: Generate Departmental Capstone Analytics & Accreditation Report . . . 51

CHAPTER 5: SYSTEM DESIGN . . . 52
5.1 Data Flow Diagram . . . 52
5.1.1 Data Flow Diagram Level-0 . . . 52
5.1.2 Data Flow Diagram Level-1 . . . 52
5.2 Sequence Diagram . . . 53
5.2.1 Sequence Diagram-01: Problem Statement Ingestion & AI Expert Recommendation . . . 53

 © Daffodil International University

5.2.2 Sequence Diagram-02: Advisory Collaboration Request & FSM Lifecycle Negotiation 54
5.2.3 Sequence Diagram-03: Expert Profile Ingestion & Dense Vector Indexing 54
5.2.4 Sequence Diagram-04: Multidisciplinary Team Skill Gap Analysis & Co-Advisor Discovery 55
5.2.5 Sequence Diagram-05: Mentorship Consultation Scheduling & Availability Check 55
5.2.6 Sequence Diagram-06: Institutional Compliance Audit Search & Export . 56
5.3 State Diagram 56
5.3.1 State Diagram-01: Advisory Request FSM Lifecycle 56
5.3.2 State Diagram-02: User Account Status 56
5.3.3 State Diagram-03: Faculty Advisory Quota & Availability State 57
5.3.4 State Diagram-04: Capstone Project Stage Lifecycle 57
5.3.5 State Diagram-05: AI Inference & Matchmaking Pipeline Status 57
5.3.6 State Diagram-06: Credential Attestation & Endorsement Status 57
5.4 Entity Relationship Diagram 57
5.5 Class Diagram 58

 © Daffodil International University

CHAPTER 1: INTRODUCTION

In modern higher education institutions, capstone projects, undergraduate dissertations, and postgraduate research represent the pinnacle of academic development. At institutions such as Daffodil International University, thousands of final-year software engineering, computer science, and engineering students embark on ambitious technical projects each semester. However, the foundational phase of these projects—identifying, evaluating, and securing qualified faculty supervisors, subject-matter experts, and industry mentors—is characterized by profound friction, systemic asymmetry, and administrative inefficiencies.

Existing university faculty directories are inherently static, departmentally siloed, and organized primarily as administrative staff rosters. These traditional repositories catalog professors solely by department, generic academic rank, and coarse keyword tags such as "Computer Networks" or "Artificial Intelligence". When undergraduate students formulate capstone ideas around emergent, specialized, or interdisciplinary challenges—such as *"Privacy-preserving federated learning architectures for intermittent low-power IoT sensor clusters"* or *"CRISPR target optimization using multi-modal deep generative transformers"*—static keyword searches fail completely. Students are unable to identify which researchers across diverse faculties possess active publication histories, practical codebase competencies, or ongoing industry grants relevant to their specific technical bottlenecks.

Consequently, students resort to blind email solicitations, informal campus word-of-mouth, and subjective peer recommendations. This dynamic imposes severe cognitive fatigue on prominent, senior faculty members who frequently receive dozens of redundant, poorly scoped advisory requests each semester. Meanwhile, specialized junior faculty, clinical researchers, and affiliated adjunct industry practitioners remain underutilized, despite possessing specialized competencies directly aligned with student project requirements. Furthermore, departmental capstone coordinators lack centralized visibility into real-time supervisory loads, leaving advisory allocations susceptible to quota imbalances, unmonitored dropouts, and accreditation compliance gaps.

UniExpert: An AI-Powered Expert Discovery and Academic Collaboration Platform is conceived to eliminate this institutional bottleneck. UniExpert replaces static keyword directories with an intelligent, problem-first matching infrastructure. Rather than forcing students to guess abstract taxonomies, the platform allows student project teams to submit unstructured, natural-language project proposals and problem descriptions. An integrated Natural Language Processing (NLP) inference pipeline extracts core domain taxonomies, prerequisite skill entities, and methodological paradigms.

These extracted attributes are transformed into high-dimensional dense embeddings and evaluated against an indexed repository of faculty research profiles, peer-reviewed publications, and verified industry mentor credentials using dense semantic vector search (Qdrant HNSW vector index). Matches are scored and ranked via a transparent, multi-criteria hybrid ranking engine that jointly weighs semantic relevance, technical skill overlap, research alignment, advisor quota availability, and institutional tier verification. Crucially, the system provides Explainable AI (XAI) justifications for every recommendation, ensuring full transparency.

The platform further incorporates an automated Multidisciplinary Skill Gap Engine that evaluates project prerequisite requirements against existing team proficiencies using a combinatorial set-cover optimization algorithm, suggesting complementary co-advisors or student peers to bridge critical knowledge deficits. To protect faculty from inbox fatigue, all collaboration solicitations are governed by an auditable Finite State Machine (FSM) backed by configurable anti-burnout supervisory quotas. Once an advisory relationship is established, UniExpert facilitates structured milestone tracking, consultation scheduling, and mutual post-project peer

endorsements. Every transaction, verification decision, and advisory milestone is permanently logged in an immutable institutional audit trail, empowering university coordinators and accreditation bodies with real-time analytics on supervisory balance, academic productivity, and multidisciplinary research velocity.

1.1 Objective

The principal objective of this capstone project is to design, implement, rigorously test, and document an enterprise-grade, AI-driven academic expert discovery and collaborative matchmaking software platform for higher education institutions. Specific technical and operational objectives include:

1. **Develop a Problem-First Semantic Discovery Engine:** Enable undergraduate and graduate students to submit raw, unstructured natural-language project proposals (ranging from short abstracts to multi-paragraph problem statements) and automatically extract domain classifications, prerequisite technical skills, and research methodologies using zero-shot classification and named entity recognition (NER).
2. **Construct a High-Performance Hybrid Matching Pipeline:** Index faculty research profiles, publication corpora, and mentor portfolios into dense vector spaces, achieving sub-second cosine semantic retrieval via Hierarchical Navigable Small World (HNSW) vector graphs combined with deterministic multi-criteria ranking (semantic score, skill overlap ratio, publication recency, verified credential weight, and real-time supervisory capacity).
3. **Implement Transparent Explainable AI (XAI) Recommendation Breakdowns:** Formulate and present granular, human-interpretable justifications for each recommended expert, showing the exact percentage contribution of domain similarity, matched skill keywords, verified publication citations, and available advisory quotas.
4. **Engineer Multidisciplinary Team Skill Gap Analysis:** Formulate a combinatorial set-cover optimization model ($\Delta\mathcal{K} = \mathcal{K}_{\text{req}} \setminus \mathcal{K}_{\text{team}}$) that identifies unaddressed prerequisite technical competencies in capstone teams and algorithmically discovers optimal co-advisors or supplementary peer researchers to maximize project viability.
5. **Govern Advisory Solicitations via Finite State Machine (FSM):** Formalize collaboration request workflows across deterministic states (*Draft*, *Submitted*, *Under Review*, *Negotiation*, *Accepted*, *Declined*, *Withdrawn*, *Completed*) with hard enforcement of configurable faculty advisory quotas and blackout schedules to eliminate faculty inbox burnout.
6. **Establish Multi-Tier Credential Verification and Peer Endorsements:** Architect a role-based verification workflow requiring institutional email domain authentication, automated Google Scholar/ORCID metadata synchronization, and mutual post-collaboration peer evaluations to maintain ecosystem reputation and integrity.
7. **Deliver Institutional Governance and Accreditation Auditing:** Build administrative monitoring dashboards with comprehensive query filtering, supervisory workload heatmaps, real-time quota tracking, and cryptographically tamper-evident CSV/PDF compliance audit logging for university evaluation committees.

1.2 Key Features

The UniExpert platform delivers a comprehensive suite of enterprise capabilities engineered specifically for university capstone and research ecosystems:

- **User Accounts and Role-Based Access Control (RBAC):** Provides secure multi-tier user registration and authentication for Students, Faculty Advisors, Industry Mentors, Department Capstone Coordinators, System Administrators, and Institutional Observers. Access permissions are strictly validated through cryptographically signed JSON Web

Tokens (JWT) with automated verification for institutional email domains (`.edu` or departmental affiliations).

- **Natural Language Problem Ingestion and Entity Parsing:** Allows students to input descriptive project statements without selecting rigid pre-configured categories. An automated NLP inference pipeline analyzes the lexical and semantic structure of the statement, extracting domain classifications (e.g., Computer Vision, Distributed Systems, Bioinformatics), prerequisite technical toolchains (e.g., PyTorch, Rust, Kubernetes), and research paradigms.
- **Dense Semantic Vector Embedding and HNSW Indexing:** Transforms unstructured student project statements and multi-source faculty publication abstracts into 384-dimensional dense semantic vectors using optimized transformer embeddings. Embeddings are indexed inside a high-throughput Qdrant vector database utilizing cosine similarity metrics on HNSW graph topologies, ensuring sub-50ms retrieval latencies across tens of thousands of academic profiles.
- **Multi-Criteria Hybrid Scoring and Ranking Engine:** Combines vector cosine similarity with multi-attribute academic signals to rank expert candidates. The hybrid scoring formulation factors in domain relevance (ω_1), exact skill overlap (ω_2), recent publication citations (ω_3), credential verification tier (ω_4), and real-time advisor calendar availability (ω_5), guaranteeing mathematically rigorous and balanced recommendations.
- **Explainable AI (XAI) Justification Dashboards:** Eliminates "black-box" recommendation opacity by generating granular visual breakdowns for every candidate match. Students and coordinators inspect radar charts and percentage attribution cards detailing why a specific expert was suggested, which project keywords triggered the match, and how the expert's past papers align with the student's methodology.
- **Multidisciplinary Team Skill Gap Optimization ($\Delta\mathcal{K}$):** Evaluates a capstone team's collective skill inventory against the prerequisite skills required by their project proposal. The system computes the unaddressed competency set ($\Delta\mathcal{K}$) and executes a set-cover optimization algorithm to suggest co-advisors or student peers from complementary departments (e.g., pairing a Software Engineering group with a Pharmacy researcher).
- **Collaboration Request Lifecycle State Machine (FSM):** Governs all student-to-expert advisory proposals through an auditable, deterministic lifecycle. Requests progress systematically through formal negotiation stages, preventing ghosting, enforcing response deadlines (e.g., 7-day auto-expiry), and ensuring formal contractual alignment between advisors and students before project kick-off.
- **Faculty Anti-Burnout Quotas and Availability Scheduling:** Protects faculty members from excessive advisory obligations by enforcing configurable per-semester active student caps (e.g., maximum 5 concurrent capstone groups). Automatically marks faculty profiles as "Busy/Quota Reached" in search results once limits are met and integrates an interactive office-hour scheduling calendar for verified advisory meetings.
- **Multi-Tier Credential Verification and Peer Endorsement:** Implements a three-tier trust architecture: Tier-1 (Institutional Email Domain Verified), Tier-2 (ORCID/Google Scholar Publication Verified), and Tier-3 (Department Chair Validated). Following capstone completion, students and faculty submit mutual structured evaluations, generating verifiable academic reputation metrics and skill endorsements.
- **Tamper-Evident Audit Trail and Accreditation Reporting:** Records every system transaction—including account approvals, search queries, advisory proposals, state transitions, and quota updates—in an append-only audit log capturing user ID, timestamp, IP address, and cryptographic event hashes. Coordinators and institutional accreditation committees export certified PDF and CSV reports on supervisory distribution, department performance, and research output.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Scenario Writing

The operational realities of modern academic institutions demand intuitive, highly responsive, and reliable software interactions. The following six real-world operational scenarios illustrate how diverse university stakeholders engage with the UniExpert platform across the end-to-end project lifecycle.

2.1.1 Scenario 1: Formulating an Unstructured Capstone Problem and Extracting Skill Entities

Scenario Description:

- A final-year Software Engineering student group logs into UniExpert and opens the "New Capstone Proposal" workspace.
- The group lead inputs a raw, three-paragraph project statement: *"We are designing an edge-computing healthcare monitoring system that collects ECG telemetry over Bluetooth Low Energy, aggregates it on microcontrollers, and runs local convolutional neural networks to detect arrhythmias before synchronizing encrypted anomalies to a cloud backend via MQTT."*
- The student clicks "Analyze Problem Formulation".
- The system's background NLP pipeline tokenizes the text, performs zero-shot transformer classification, and identifies primary domains: `Embedded Systems`, `Machine Learning`, and `Healthcare Informatics`.
- The entity extractor tags prerequisite technical competencies: `BLE (Bluetooth Low Energy)`, `C/C++ Embedded`, `CNN (Convolutional Neural Networks)`, `MQTT`, and `Data Encryption (AES-GCM)`.
- The system displays the extracted domains and skills on an interactive review card, allowing the team lead to adjust importance sliders (e.g., setting `CNN` to high importance and `MQTT` to medium).
- The student confirms the extracted entity set, saving the parsed project vector to the repository and initiating candidate expert matching.

2.1.2 Scenario 2: Semantic Hybrid Expert Discovery and Reviewing XAI Explanations

Scenario Description:

- Following entity extraction, the student triggers the "Discover Advisors" query.
- The matchmaking engine converts the project statement into a 384-dimensional dense semantic embedding and queries the Qdrant HNSW vector store across indexed faculty profiles.
- The hybrid ranking algorithm calculates multi-criteria composite scores across semantic closeness, verified skill overlap, publication recency, and current supervisor quota capacity.
- The search interface presents a ranked list of recommended professors and industry practitioners, each displaying an overall match score (e.g., "94% Match").
- The student expands the top-ranked candidate profile (Dr. Farhana Rahman, Associate Professor of Embedded AI).
- The Explainable AI (XAI) drawer displays a detailed breakdown: 45% contribution from direct semantic vector similarity with her recent IEEE papers on edge inference, 30% from verified skill matches (`C/C++`, `Edge ML`), 15% from Tier-2 verified credentials, and 10%

from active advisory quota availability (2 out of 5 slots remaining).

- The student benchmarks Dr. Rahman side-by-side with a second candidate to compare available consultation hours, departmental affiliations, and active research grants before proceeding.

2.1.3 Scenario 3: Detecting Multidisciplinary Skill Gaps and Recruiting Co-Advisors

Scenario Description:

- A student team comprising three Computer Science majors inputs a capstone project involving bioinformatics: *"Deep learning pipeline for single-cell RNA-seq clustering and genomic variant classification."*
- The team profile indicates member proficiencies in `Python`, `PyTorch`, and `Cloud Computing`.
- The system's Skill Gap Detection Engine calculates the competency deficit set: $\Delta\mathcal{K} = \mathcal{K}_{\text{req}} \setminus \mathcal{K}_{\text{team}} = \{\texttt{Single-Cell RNA-seq}, \texttt{Genomic Annotation}, \texttt{Bioinformatics Pipelines}\}$.
- The platform displays a prominent warning badge: *"Critical Domain Deficit Detected: 60% of biological prerequisites unaddressed by current team roster."*
- The system triggers a combinatorial set-cover optimization algorithm, evaluating faculty members in the Department of Pharmacy and Department of Public Health.
- UniExpert recommends Dr. Tanvir Ahmed (Bioinformatics Researcher) as a prospective Co-Advisor, highlighting that adding Dr. Ahmed increases total project competency coverage to 92%.
- The student group clicks "Propose Co-Advisory", linking Dr. Ahmed alongside their primary software engineering supervisor.

2.1.4 Scenario 4: Negotiating Collaboration Requests via Finite State Machine

Scenario Description:

- After selecting Dr. Farhana Rahman, the student team clicks "Request Advisory Collaboration".
- The team completes a structured proposal form specifying proposed project milestones, weekly expected time commitment, and a brief justification referencing Dr. Rahman's edge computing publications.
- The system creates an advisory request record, transitions the state machine from `DRAFT` to `SUBMITTED`, and delivers an instant dashboard notification and email digest to Dr. Rahman.
- Dr. Rahman logs into her Faculty Advisory Portal and inspects the incoming request in her "Pending Solicitations" queue.
- Rather than an open-ended generic email, she reviews the structured milestones, project synopsis, and the system's XAI alignment score.
- She selects "Request Clarification / Negotiation", submitting a condition: *"Scope needs to benchmark TinyML on ARM Cortex-M4 hardware rather than simulated devices."*
- The FSM transitions to `NEGOTIATION`. The students receive the note, update their milestone roadmap, and re-submit.
- Dr. Rahman reviews the revised roadmap and clicks "Accept Advisory Request". The FSM transitions to `ACCEPTED`, automatically decrementing her available quota from 2 to 1 and notifying the Department Coordinator.

2.1.5 Scenario 5: Managing Faculty Quota Exhaustion and Scheduling Office Hours

Scenario Description:

- A highly sought-after Machine Learning professor reaches his departmental supervisory quota of 5 concurrent capstone groups for the Fall 2026 term.

- The system automatically transitions his availability status from `AVAILABLE` to `QUOTA_REACHED`.
- In general student search results, his profile card is greyed out with a badge indicating *"Supervisory Quota Full for Current Semester (Co-Advisory or Consultation Only)"*, blocking new primary advisory submissions and preventing inbox flooding.
- For his five active student groups, the professor opens his "Advisory Calendar Settings" and defines available weekly office hour slots (Tuesdays 2:00 PM – 4:00 PM).
- The student team lead opens the professor's shared scheduling view, selects an open 30-minute consultation window for next Tuesday, and inputs a session agenda: *"Midterm architecture review and sensor calibration validation."*
- The system validates the booking against the professor's calendar, generates a calendar event with a secure meeting link, and logs the scheduled consultation in the team's capstone workspace.

2.1.6 Scenario 6: Verifying Academic Credentials and Generating Compliance Audit Reports

Scenario Description:

- A newly recruited adjunct faculty member registers on UniExpert with a university email address.
- The system checks the email domain, verifies institutional ownership via a 24-hour cryptographic token, and promotes the profile to Tier-1 (Email Verified).
- The faculty member connects their ORCID and Google Scholar ID. The system's background ingestion worker queries academic APIs, parses 18 indexed publications, computes an h-index of 7, and upgrades the account to Tier-2 (Scholar Verified).
- At the end of the academic semester, the Department Capstone Coordinator accesses the Administrative Compliance Console.
- The coordinator reviews department-wide supervisory distribution heatmaps, verifying that no faculty member exceeded their statutory quota and that all 142 registered capstone groups were paired with verified advisors.
- The coordinator applies filters for the "Department of Software Engineering, Fall 2026", verifies all cryptographic audit log hashes, and generates a signed PDF/CSV Accreditation Compliance Report for the University Academic Council.

2.2 Stakeholders

The UniExpert platform orchestrates collaborative interactions among six primary academic stakeholders:

1. **Student Project Lead / Capstone Researcher:** Undergraduate or postgraduate students undertaking capstone projects, academic theses, or competitive innovation challenges. They utilize the platform to formulate problem statements, receive AI-driven expert recommendations, analyze team skill deficits, submit formal advisory requests, schedule consultations, and maintain milestone communication.
2. **Faculty Advisor / Academic Expert:** University professors, associate professors, assistant professors, and departmental lecturers who supervise student research. They define their research taxonomies, sync publication archives, configure weekly advisory quotas and blackout dates, evaluate structured student proposals through the FSM lifecycle, and conduct supervised milestone reviews.
3. **Industry Mentor / Technical Practitioner:** Verified external domain professionals, software architects, corporate research scientists, and alumni fellows who contribute specialized industry perspectives, code reviews, and architectural guidance to academic capstone teams.

4. **Department Capstone Coordinator:** Academic faculty appointed to manage departmental capstone course modules. They oversee group registrations, establish supervisory quota limits, adjudicate interdisciplinary co-advisory agreements, monitor faculty workload equity, and resolve advisory disputes or unallocated student teams.
5. **System Administrator:** Technical personnel (represented by the engineering development team during initial deployment) responsible for platform operations, database migrations, Qdrant vector store indexing, LLM inference pipeline tuning, role authorization governance, and security backups.
6. **Institutional Auditor / Accreditation Observer:** University executive leadership, Dean's office personnel, Board of Studies evaluators, and external accreditation assessment committees (e.g., BAETE/IEB) who view aggregated supervisory metrics, track institutional research diversity, and audit compliance trails without modifying transactional data.

2.3 User Profiles

The structured profiles below define the operational characteristics, demographic assumptions, technical experience, and derived engineering requirements for the core user classes of UniExpert.

2.3.1 User Profile-01: Student Project Lead

User Class	Notes on Characteristic	Requirement Implied
Type of User	Undergraduate / Graduate student undertaking capstone project	Verification, Identity Management
Age Range	Approximately 20 to 28 years	Interface, Accessibility
Frequency of Use	Daily during proposal setup and weekly during capstone execution	Performance, Reliability, Operational
Mandatory / Discretionary	Mandatory for capstone team leads; discretionary for general thesis exploration	Interface, Usability, Acceptance
Computer Experience / OS / Applications	High digital fluency; familiar with Git, web browsers, IDEs, and mobile applications	Modern Web UI, Responsive Design, Portability
Education	3rd / 4th-year university undergraduate or Master's candidate	Technical Language Support, Documentation
Goals	Find optimal supervisors, resolve skill deficits, and track advisory milestones	Reliability, Recommendation Accuracy, XAI
Language Skills	English (primary academic medium) and Bangla (informal communication)	UTF-8 Text Support, English UI
Number of Users	500 to 2,500 active students per semester	Scalability, Concurrency, Resource Management
Training	10-minute briefing during capstone orientation lecture	Intuitive Onboarding, Contextual Tooltips
Other Systems Used	Google Classroom, GitHub, Moodle, Overleaf, WhatsApp, Email	Import/Export, Calendar Integration
Ways of Working	Collaborative laptop environments, campus Wi-Fi, home internet	Session Resiliency, Responsive Web

Table 2.1: User Profile for Student Project Lead

2.3.2 User Profile-02: Faculty Advisor / Academic Expert

User Class	Notes on Characteristic	Requirement Implied
Type of User	Full-time university faculty, professors, and academic researchers	Role-Based Security, Institutional Domain Check
Age Range	Approximately 28 to 65 years	Clean Typography, High Contrast UI
Frequency of Use	2 to 4 times weekly for reviewing proposals and consultation logs	Efficiency, Email Notification Digest
Mandatory / Discretionary	Mandatory for assigned supervisors; discretionary for elective co-advisors	Quota Enforcement, Anti-Burnout Protection
Computer Experience / OS / Applications	Advanced academic software proficiency, Google Scholar, LaTeX, ORCID	Automated Publication Scraping, ORCID OAuth
Education	M.Sc., M.Phil., Ph.D. in engineering or allied academic fields	High-Level Academic Taxonomy Support
Goals	Supervise aligned research, prevent inbox overload, and track group milestones	Finite State Machine, Structured Proposals
Language Skills	Fluent academic English	Strict Technical English Formatting
Number of Users	80 to 300 faculty members per institution	Rapid Indexing, Database Optimization
Training	15-minute hands-on departmental orientation or self-guided manual	Quick-Start Guides, Low Friction UI
Other Systems Used	University ERP, Microsoft Teams, Zoom, Google Calendar, ResearchGate	Calendar Sync, Webhook Integrations
Ways of Working	Office desktop PCs, personal laptops, mobile browsers during transit	Cross-Browser Stability, Minimalist Dashboard

Table 2.2: User Profile for Faculty Advisor

2.3.3 User Profile-03: Industry Mentor

User Class	Notes on Characteristic	Requirement Implied
Type of User	External industry technical leads, corporate researchers, and alumni	External Verification, LinkedIn Integration
Age Range	Approximately 25 to 55 years	Modern Ergonomic Web Interface
Frequency of Use	1 to 2 times weekly during designated mentorship sprints	Rapid Async Workflows, Push Notifications
Mandatory / Discretionary	Discretionary (voluntary industry-academia engagement)	Frictionless Sign-In, Flexible Availability
Computer Experience / OS / Applications	Professional corporate software engineering environments	Security Standards, Clean RESTful APIs
Education	B.Sc. / M.Sc. in Computer Science, Software Engineering, or related	Industry Skill Taxonomy (Cloud, DevOps, AI)
Goals	Guide innovative student prototypes, identify hiring talent, and give back	Structured Advisory Scopes, Direct Booking
Language Skills	English and Bangla professional fluency	Bi-Lingual Form Inputs
Number of Users	30 to 150 registered industry mentors	Controlled External Access, Privacy
Training	Self-guided digital onboarding walkthrough (<5 minutes)	Intuitive UX, Self-Explanatory Forms
Other Systems Used	Slack, GitHub, Jira, LinkedIn, Google Meet	GitHub Profile Sync, Meeting Integrations
Ways of Working	Work laptops, mobile devices during business hours	Responsive UI, Session Token Persistence

Table 2.3: User Profile for Industry Mentor

2.3.4 User Profile-04: Department Capstone Coordinator

User Class	Notes on Characteristic	Requirement Implied
Type of User	Senior faculty member appointed as Capstone Course Convener	Elevated Administrative RBAC Scopes
Age Range	Approximately 32 to 55 years	High-Density Data Tables, Visual Charts
Frequency of Use	Heavy daily use at semester start; 2 to 3 times weekly thereafter	High Data Throughput, Batch Operations
Mandatory / Discretionary	Mandatory departmental governance responsibility	Strict Auditability, Data Validation
Computer Experience / OS / Applications	Highly experienced with spreadsheets, university portals, and CSV exports	Robust CSV/Excel Batch Importer, PDF Reports
Education	Ph.D. or Senior Assistant / Associate Professor	Formal Departmental Workflow Modeling
Goals	Ensure 100% student team supervisor coverage, balance faculty loads, audit ethics	Quota Monitoring Dashboards, Allocation Heatmaps
Language Skills	Fluent academic and administrative English	Formal Academic Reporting
Number of Users	2 to 6 coordinators per department	Granular Authorization, State Overrides
Training	Dedicated administrative briefing and technical handover session	Comprehensive Administrative Manual
Other Systems Used	University Examination System, Institutional Spreadsheets	Data Interoperability, Secure Export
Ways of Working	Office desktop with high-resolution monitors, campus LAN	Wide Table Views, Multi-Column Layouts

Table 2.4: User Profile for Department Capstone Coordinator

2.3.5 User Profile-05: System Administrator

User Class	Notes on Characteristic	Requirement Implied
Type of User	Core software engineering team and university IT administrators	Superuser Privileges, System Logs Access
Age Range	Approximately 22 to 45 years	CLI Tools, Configuration Consoles
Frequency of Use	Continuous background monitoring; intensive during deployment	Health Telemetry, Automated Alerts
Mandatory / Discretionary	Mandatory platform maintenance responsibility	High Availability, Disaster Recovery
Computer Experience / OS / Applications	Expert; Linux servers, Docker containers, PostgreSQL, Qdrant, Redis	Containerization, CI/CD, Infrastructure as Code
Education	B.Sc. / M.Sc. in Software Engineering or Computer Science	Deep Software Architecture Mastery
Goals	Maintain 99.5% system uptime, optimize vector query latencies, secure data	Log Auditing, Metric Dashboards, DB Backups
Language Skills	English (Technical)	Structured Logging, Standard Error Codes
Number of Users	1 to 3 primary administrators	Multi-Factor Auth, Cryptographic Key Management
Training	Architectural blueprints, API documentation, runbooks	Technical Documentation Integrity
Other Systems Used	Docker, Kubernetes, Grafana, PostgreSQL CLI, Qdrant Web UI	Metrics Exporters, System Health Endpoints
Ways of Working	Secure terminal sessions, administrative web consoles	Strict HTTPS, IP Whitelisting, CORS

Table 2.5: User Profile for System Administrator

2.4 Scope

The UniExpert platform encompasses ten core operational modules designed to automate, optimize, and govern the academic matchmaking and collaboration lifecycle:

1. **User Registration, Role-Based Access Control, and Institutional Identity Verification:**

(a) *Objective:* Establish an authentic, role-restricted user registry ensuring academic integrity across students, faculty, and industry practitioners.

(b) *Functionality:* Users register with university email addresses, full legal names, departmental affiliations, and secure passwords. Faculty accounts automatically trigger cryptographic institutional token challenges to verify `.edu` domains. Strict RBAC assigns permissions across six designated scopes. All web traffic is protected via short-lived JWTs and encrypted refresh tokens.

2. Faculty Profile Ingestion, Semantic Indexing, and Publication Sync:

(a) *Objective:* Maintain comprehensive, dynamic academic expert profiles without requiring repetitive manual data entry from professors.

(b) *Functionality:* Ingests curriculum vitae data, research areas, and publication archives from Google Scholar and ORCID APIs. Automatically aggregates paper abstracts, citation indices, and co-author graphs into structured JSON representations and prepares text corpora for dense vector embedding.

3. Problem-First Natural Language Project Parsing and Skill Extraction:

(a) *Objective:* Enable student teams to express research objectives naturally without being constrained by predetermined keyword checklists.

(b) *Functionality:* Accepts multi-paragraph problem statements, abstracts, or methodology outlines. Deploys zero-shot transformer classifiers and Named Entity Recognition (NER) models to extract macro research domains, technical toolchains, algorithms, and methodological paradigms, generating editable entity tags with user-adjustable relevance weights.

4. Dense Semantic Vector Search and Multi-Criteria Hybrid Matchmaking:

(a) *Objective:* Discover the most qualified supervisors and mentors through deep semantic understanding combined with objective academic signals.

(b) *Functionality:* Encodes project descriptions into 384-dimensional dense vectors using transformer embeddings. Executes approximate nearest neighbor (ANN) vector search across Qdrant HNSW graph collections using cosine similarity. Combines cosine scores with skill overlap ratios, citation recency weights, credential tier multipliers, and advisor availability into a normalized composite score ($S_{\text{hybrid}} \in [0, 100]$).

5. Explainable AI (XAI) Justifications and Match Decomposition:

(a) *Objective:* Foster trust and transparency by explaining exactly why an expert was recommended.

(b) *Functionality:* Displays interactive visual breakdown cards for every recommendation. Shows percentage contributions of domain alignment, exact keyword intersections, matching past publication titles, and current advisory bandwidth.

6. Multidisciplinary Gap Detection and Set-Cover Team Optimization:

(a) *Objective:* Identify missing technical proficiencies in student groups and recommend cross-departmental co-advisors or peer members.

(b) *Functionality:* Compares project prerequisite competencies ($\mathcal{K}_{\text{req}}$) with student team proficiencies ($\mathcal{K}_{\text{team}}$) to derive the competency deficit set ($\Delta\mathcal{K}$). Solves a combinatorial set-cover optimization problem to identify the minimal set of co-advisors or student peers required to cover 100% of project prerequisites.

7. Collaboration Advisory Request Lifecycle State Machine (FSM):

(a) *Objective:* Replace chaotic email solicitations with a formal, trackable advisory negotiation workflow.

(b) *Functionality:* Enforces a formal state machine governing proposals: `DRAFT` $\rightarrow$

`SUBMITTED` → `UNDER_REVIEW` → `NEGOTIATION` → `ACCEPTED` / `DECLINED` / `EXPIRED`. Requests carry structured milestone proposals, enforce 7-day auto-response timers, and prevent duplicate concurrent applications.

8. **Smart Consultation Scheduling and Contextual Workspace Channels:**
   (a) *Objective:* Streamline faculty-student consultation bookings and centralize capstone progress communications.
   (b) *Functionality:* Enables advisors to publish weekly availability windows and maximum daily consultation limits. Students book 30-minute consultation slots with pre-attached discussion agendas. Provides an integrated, audit-logged message channel for milestone document exchanges.
9. **Mutual Peer Endorsement, Credential Attestation, and Reputation Scoring:**
   (a) *Objective:* Maintain qualitative accountability and recognize exceptional mentorship across the institution.
   (b) *Functionality:* Prompts students and advisors to submit structured qualitative evaluations and quantitative rubric ratings upon capstone milestone completion. Attests verified skill proficiencies to student portfolios and updates faculty mentorship reputation scores.
10. **System Audit Logging, Administrative Moderation, and Institutional Analytics Reporting:**
    (a) *Objective:* Provide department heads and accreditation evaluators with full visibility, security governance, and compliance reporting.
    (b) *Functionality:* Appends all system events to an immutable audit log. Generates real-time supervisory allocation heatmaps, department load distributions, and downloadable PDF/CSV accreditation reports compliant with university committee standards.

2.4.1 Scope Boundaries

To maintain strict architectural feasibility and focus on core software engineering deliverables within the capstone timeframe, the following boundaries are formally established:

- **No Automated Thesis Grading:** UniExpert facilitates expert discovery, advisory allocation, and milestone scheduling; it does not perform automated grading, plagiarism detection, or academic evaluation of submitted thesis reports.
- **No Financial Transactions or Honorarium Processing:** The platform manages academic mentorship engagements and does not incorporate payment gateways, honorarium disbursements, or sponsorship billing.
- **No Replacement of Official University LMS/ERP:** UniExpert operates as a specialized matchmaking and supervisory allocation sidecar; it does not replace official institutional gradebook registers, student admission portals, or university fee collection systems.
- **No Unmoderated Social Chatting:** Messaging channels are strictly tied to formal active advisory workspaces and milestone submissions to preserve professional academic decorum.
- **English Language Domain Focus:** While user interfaces accommodate standard UTF-8 characters and Bengali naming conventions, the core NLP transformer pipelines for taxonomy extraction and semantic vector embeddings are optimized for English academic literature.

2.5 Feasibility Study

A rigorous multi-dimensional feasibility study was conducted to verify that the UniExpert platform can be successfully architected, developed, evaluated, and deployed within university

capstone constraints.

1. **Technical Feasibility:**
   (a) *Hardware and Infrastructure:* The development and execution environments utilize standard, accessible consumer and academic computing hardware. Client frontends operate smoothly on standard web browsers (Chrome, Firefox, Safari, Edge) across laptops and mobile devices. Backend microservices, the PostgreSQL relational database, Redis caching layers, and the Qdrant vector database require only 4 CPU cores and 8 GB RAM on cloud virtual private servers or local university lab machines.
   (b) *Software and Technical Approach:* The system leverages robust, industry-standard open-source technologies: Next.js and TypeScript for the responsive reactive frontend, FastAPI (Python) for asynchronous REST endpoints, FastEmbed/HuggingFace transformers for localized high-efficiency vector embeddings, and Qdrant for vector indexing. Localized embedding inference avoids third-party API rate limits and external recurring costs.
   (c) *Technical Expertise:* The project team possesses core software engineering proficiencies in full-stack web development, relational database modeling, containerization, and modern machine learning frameworks acquired through the Software Engineering curriculum.
2. **Operational Feasibility:**
   (a) *User Adoption:* Adoption is driven by addressing an acute, recognized institutional pain point—faculty email fatigue and student matchmaking friction. A streamlined onboarding workflow (institutional email verification), automated publication scraping (eliminating manual profile entry), and clean XAI justifications ensure high usability across students, faculty, and administrators.
   (b) *Impact on Current Academic Practice:* The platform replaces informal, unmonitored email threads and static spreadsheets with an auditable, transparent advisory state machine. It introduces no disruptive bureaucratic overhead, significantly reduces supervisor inbox clutter, and balances department workloads.
3. **Economic Feasibility:**
   (a) *Development Cost:* The platform is constructed entirely using open-source frameworks, free academic tiers, and permissive MIT/Apache-2.0 licensed software. No commercial proprietary licenses are required.
   (b) *Operating Cost and Sustainability:* Running the localized embedding pipeline (FastEmbed `all-MiniLM-L6-v2`) and self-hosted Qdrant container consumes minimal server resources, incurring nominal hosting fees ($10–$20/month on standard cloud VPS or $0 on internal university server racks).
4. **Scheduling Feasibility:**
   (a) *Project Timeline:* The 12-week Fall 2026 capstone schedule is structured across disciplined agile sprints: Requirements & System Analysis (Weeks 1–3), System Architecture & Database Modeling (Weeks 4–5), Core Engine & Vector Search Development (Weeks 6–8), Collaboration FSM & Admin Dashboards (Weeks 9–10), and Rigorous Verification, IR Benchmarking & Documentation (Weeks 11–12).
   (b) *Dependencies:* The project operates independently of external institutional approvals or commercial API paywalls by utilizing synthetic academic datasets, open-access publication metadata (ORCID/arXiv), and isolated containerized testbeds.
5. **Security and Privacy Considerations:**
   (a) *Data Protection:* Student and faculty personal identifying information (PII) is protected via AES-256 database field encryption, salted Argon2 password hashing, and

encrypted TLS 1.3 transport. Role-Based Access Control guarantees that student project drafts remain private to authorized team members and assigned mentors.

(b) *Legal and Ethical Considerations:* The architecture conforms to institutional academic integrity standards and data privacy principles (e.g., GDPR data minimization). Users retain explicit control over profile visibility, and comprehensive immutable audit logs track all data access events.

Feasibility Summary: The UniExpert system is technically, economically, operationally, and legally feasible. The combination of localized open-source AI tooling, modular microservice architecture, and clear domain boundaries ensures high reliability and successful implementation within the academic capstone timeframe.

2.6 Project Block Diagram

The high-level operational flows and functional boundaries for each primary user role are detailed in the five architectural block diagrams below.

2.6.1 Block Diagram-1: Student Project Lead

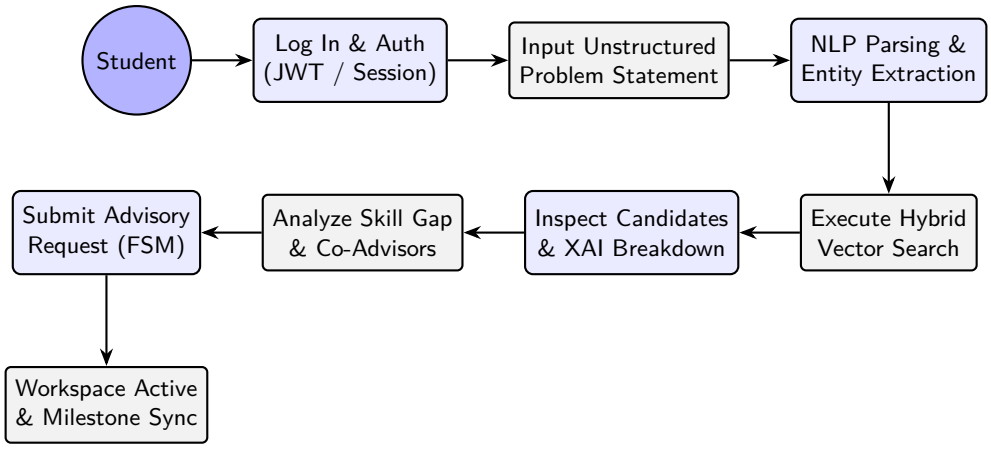


Figure 2.1: Block Diagram for Student Project Lead

2.6.2 Block Diagram-2: Faculty Advisor

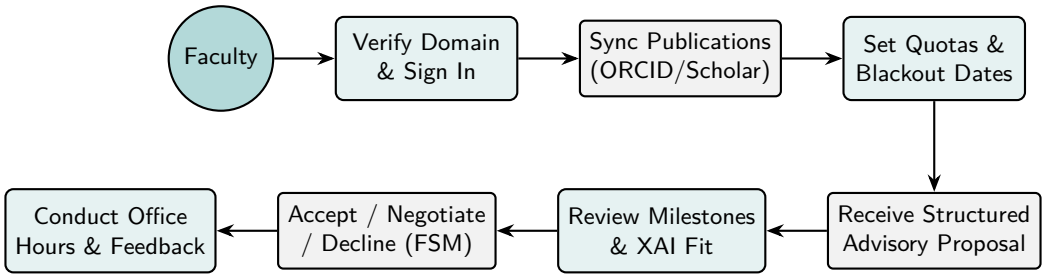


Figure 2.2: Block Diagram for Faculty Advisor

2.6.3 Block Diagram-3: Industry Mentor

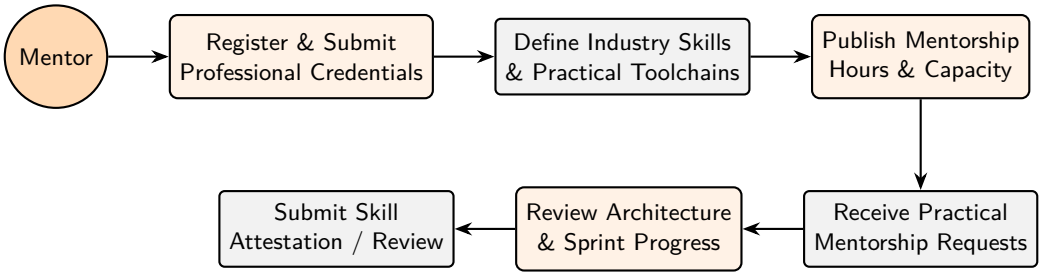


Figure 2.3: Block Diagram for Industry Mentor

2.6.4 Block Diagram-4: Department Capstone Coordinator

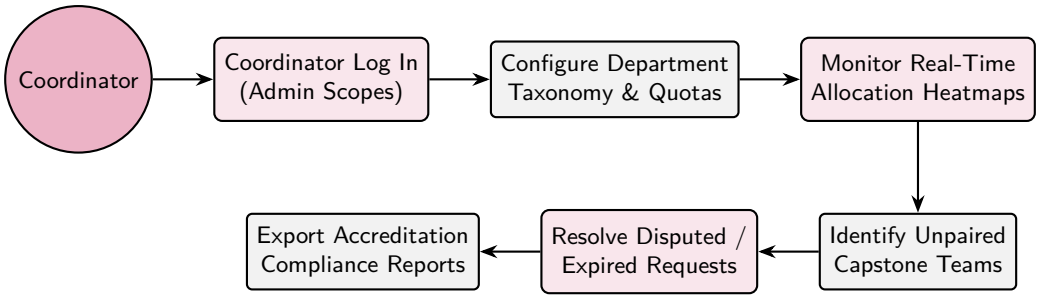


Figure 2.4: Block Diagram for Department Capstone Coordinator

2.6.5 Block Diagram-5: System Administrator

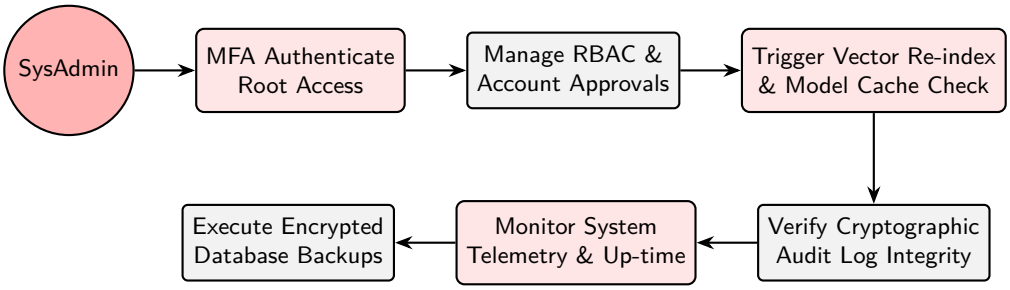


Figure 2.5: Block Diagram for System Administrator

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

The functional requirements (FRs) define the specific capabilities, user actions, system responses, and operational workflows implemented within the UniExpert platform.

FR01	**User Registration & Institutional Verification**
Description	The system SHALL allow users to register with full name, university email, department, role, and password. Faculty registering with institutional domains (`.edu`) trigger an automated cryptographic verification challenge token.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor, System Administrator

FR02	**Multi-Factor Authentication & Session Management**
Description	The system SHALL authenticate users using salted password hashing, issue short-lived JWT access tokens (15-min TTL) and HTTP-only refresh tokens (7-day TTL), and enforce multi-factor authentication (MFA) for administrative accounts.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor, Department Coordinator, System Administrator

FR03	**Student Profile & Repository Management**
Description	The system SHALL enable students to manage personal profiles including degree program, student ID, technical skills, GitHub project links, curriculum vitae (CV) uploads, and portfolio descriptions.
Stakeholder	Student Project Lead, Department Capstone Coordinator

FR04	**Faculty Expert Profile & Publication Sync**
Description	The system SHALL automatically ingest faculty publication archives, abstracts, citation counts, and h-indices by connecting to Google Scholar and ORCID APIs, compiling a comprehensive research corpus.
Stakeholder	Faculty Advisor, Department Capstone Coordinator, System Administrator

FR05	**Multi-Tier Academic Credential Verification**
Description	The system SHALL assign hierarchical verification badges: Tier-1 (Institutional Email Domain Verified), Tier-2 (ORCID/Scholar Scraped & Verified), and Tier-3 (Department Head / Committee Validated).
Stakeholder	Faculty Advisor, Industry Mentor, Department Capstone Coordinator, System Administrator

FR06	**Department & Academic Taxonomy Management**
Description	The system SHALL maintain a hierarchical tree of academic disciplines, departments, research specializations, and prerequisite technical skills, modifiable by authorized administrators without software redeployment.
Stakeholder	Department Capstone Coordinator, System Administrator

FR07	Natural Language Problem Ingestion
Description	The system SHALL provide a multi-paragraph input interface accepting unstructured capstone problem statements, project abstracts, and proposed methodologies up to 5,000 characters in length.
Stakeholder	Student Project Lead

FR08	Automated Domain & Skill Entity Extraction
Description	The system SHALL process ingested problem statements via zero-shot transformer classification and NER to extract macro academic domains, specific technical skill entities, and methodological paradigms.
Stakeholder	Student Project Lead

FR09	Interactive Requirement Refinement & Weighting
Description	The system SHALL display extracted domain and skill entities in an interactive review card, allowing students to add missing entities, remove irrelevant tags, and adjust relative importance weights.
Stakeholder	Student Project Lead

FR10	Dense Semantic Vector Embedding & Indexing
Description	The system SHALL generate 384-dimensional dense semantic vector embeddings for problem statements and faculty profiles, indexing them inside the Qdrant HNSW vector store using cosine distance metrics.
Stakeholder	Student Project Lead, System Administrator

FR11	Multi-Criteria Hybrid Match Scoring
Description	The system SHALL rank candidate experts using a composite scoring function weighting vector semantic similarity (ω_1), exact skill overlap (ω_2), publication recency (ω_3), credential tier (ω_4), and quota capacity (ω_5).
Stakeholder	Student Project Lead, Faculty Advisor

FR12	Explainable AI (XAI) Justification Generation
Description	The system SHALL provide an explainability breakdown for each match, displaying percentage attributions, matched publication titles, shared skill keywords, and supervisory quota status.
Stakeholder	Student Project Lead, Faculty Advisor, Department Capstone Coordinator

FR13	Dynamic Faceted Filtering & Availability Search
Description	The system SHALL permit students to dynamically filter candidate matches by faculty department, verification tier, available advisory capacity, and external industry status.
Stakeholder	Student Project Lead

FR14	Side-by-Side Candidate Comparison Matrix
Description	The system SHALL allow users to select up to three candidate experts and display a side-by-side comparative matrix evaluating skills, citation counts, active advisory loads, and past student ratings.
Stakeholder	Student Project Lead

FR15	Multidisciplinary Project Decomposition
Description	The system SHALL analyze complex project statements to detect multi-departmental requirements (e.g., Software Engineering + Biomedical Science) and categorize prerequisite skill clusters.
Stakeholder	Student Project Lead, Department Capstone Coordinator

FR16	Set-Cover Team Skill Gap Optimization
Description	The system SHALL compute unaddressed prerequisite skills ($\Delta\mathcal{K} = \mathcal{K}_{\text{req}} \setminus \mathcal{K}_{\text{team}}$) and execute a greedy set-cover algorithm to recommend complementary co-advisors or student peers.
Stakeholder	Student Project Lead, Department Capstone Coordinator

FR17	Collaboration Request Advisory FSM
Description	The system SHALL enforce an auditable state machine governing student proposals (`DRAFT` → `SUBMITTED` → `UNDER_REVIEW` → `NEGOTIATION` → `ACCEPTED` / `DECLINED` / `EXPIRED`) with a 7-day timeout.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor

FR18	Faculty Anti-Burnout Quota & Blackout Management
Description	The system SHALL allow faculty to define maximum active capstone supervision quotas (default: 5 groups) and blackout dates, automatically blocking new proposals when capacity is exhausted.
Stakeholder	Faculty Advisor, Department Capstone Coordinator

FR19	Consultation Booking & Calendar Synchronization
Description	The system SHALL provide an interactive office-hour scheduling calendar allowing approved student teams to book 30-minute advisory consultations with pre-attached agendas.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor

FR20	Contextual Discussion & Milestone Messaging
Description	The system SHALL provide dedicated, secure discussion threads for each active project workspace, enabling milestone document sharing, feedback logging, and version review.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor

FR21	Mutual Post-Collaboration Peer Endorsement
Description	The system SHALL solicit structured quantitative ratings and qualitative rubric reviews from students and faculty upon project completion, updating public skill attestations and mentor reputation.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor

FR22	Departmental Supervisory Allocation Dashboard
Description	The system SHALL provide coordinators with real-time supervisory allocation heatmaps, identifying unallocated student groups, overloaded professors, and quota distribution across departments.
Stakeholder	Department Capstone Coordinator, Institutional Auditor

FR23	In-App & Email Event Notifications
Description	The system SHALL trigger real-time in-app badge alerts and transactional email digests for incoming proposals, state machine transitions, consultation reminders, and quota updates.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor, Department Coordinator

FR24	Immutable System Audit Trail Recording
Description	The system SHALL log every user login, profile update, search query, proposal state transition, and quota change into an append-only audit ledger with user ID, timestamp, and IP address.
Stakeholder	System Administrator, Institutional Auditor

FR25	Audit Log Search & Regulatory Data Export
Description	The system SHALL allow authorized administrators and auditors to filter the audit trail by date range, user ID, event type, and outcome, and export verified records as CSV or encrypted JSON.
Stakeholder	Department Capstone Coordinator, System Administrator, Institutional Auditor

FR26	Institutional Accreditation & Analytics Reporting
Description	The system SHALL generate certified PDF and CSV summary reports detailing departmental supervision metrics, interdisciplinary collaboration frequency, and milestone completion velocity.
Stakeholder	Department Capstone Coordinator, System Administrator, Institutional Auditor

3.2 Non-Functional Requirements

The non-functional requirements (NFRs) specify the qualitative constraints, performance thresholds, security baselines, and architectural standards enforced across the system.

3.2.1 Security

NFR01	Role-Based Access Control (RBAC)
Description	The system SHALL strictly enforce authorization boundaries across all 6 roles, verifying JWT bearer tokens and claims on every backend API invocation.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor, Department Coordinator, System Administrator, Institutional Auditor

NFR02	Token-Based Session Security & Cryptographic Expiry
Description	Access tokens SHALL expire after 15 minutes of inactivity; refresh tokens SHALL be stored exclusively in secure, HttpOnly, SameSite cookies with a 7-day maximum lifespan.
Stakeholder	All Stakeholders

NFR03	Password Hashing & Brute-Force Lockout
Description	User passwords SHALL be hashed using Argon2id with unique cryptographic salts. Accounts SHALL be temporarily locked for 15 minutes after 5 consecutive failed login attempts.
Stakeholder	All Stakeholders

NFR04	Transport and Storage Cryptography
Description	All communication across client and microservice boundaries SHALL mandate TLS 1.3 encryption. Sensitive database fields (tokens, CV attachments, private contact numbers) SHALL use AES-256-GCM.
Stakeholder	System Administrator

3.2.2 Privacy

NFR05	Academic Data Minimization & Consent
Description	The platform SHALL collect only academic metadata strictly necessary for match-making. Student proposals SHALL remain private until the team author explicitly submits for advisory review.
Stakeholder	Student Project Lead, Faculty Advisor

NFR06	Restricted Student/Faculty PII Visibility
Description	Personal contact details (phone numbers, private emails) SHALL be masked from public search results and revealed only between confirmed collaboration partners.
Stakeholder	Student Project Lead, Faculty Advisor, Industry Mentor

NFR07	Data Retention & Automated Archival Purge
Description	Completed project workspace artifacts and temporary proposal attachments SHALL be archived 180 days following formal academic semester closure, with full audit trail metadata retained.
Stakeholder	Department Capstone Coordinator, System Administrator

3.2.3 Reliability

NFR08	Continuous System Availability & Fault Tolerance
Description	The platform SHALL maintain 99.5% service uptime during active academic semesters, supported by automated Docker health checks and container auto-restarts.
Stakeholder	All Stakeholders

NFR09	Idempotent Advisory Request Handling
Description	All collaboration state transitions SHALL be strictly idempotent using unique UUIDs, preventing double-acceptance or duplicate quota decrementing across network retries.
Stakeholder	Student Project Lead, Faculty Advisor

NFR10	Relational-Vector Eventual Consistency
Description	Profile modifications committed to PostgreSQL SHALL be asynchronously propagated and re-indexed inside Qdrant within 10 seconds under standard operating loads.
Stakeholder	System Administrator

NFR11	Fault-Tolerant AI Engine Fallback
Description	In the event of dense vector embedding service unavailability, the system SHALL automatically degrade to exact-match keyword BM25 retrieval without throwing runtime exceptions.
Stakeholder	Student Project Lead, System Administrator

NFR12	Data Integrity and Audit Traceability
Description	Audit logs SHALL be append-only and cryptographically chained; database constraints SHALL reject any UPDATE or DELETE queries executed against audit tables.
Stakeholder	System Administrator, Institutional Auditor

NFR13	Automated Backup & Disaster Recovery
Description	PostgreSQL database snapshots and Qdrant collections SHALL undergo automated daily incremental backups, with full disaster recovery verification tested each semester.
Stakeholder	System Administrator

3.2.4 Performance

NFR14	Sub-Second Vector Retrieval Latency
Description	Semantic nearest-neighbor vector search queries across 10,000 indexed faculty profiles SHALL return top-20 candidate results within 200 milliseconds (p95).
Stakeholder	Student Project Lead

NFR15	Hybrid Match Scoring Throughput
Description	The multi-criteria ranking engine SHALL compute composite scores, XAI percentage breakdowns, and availability multipliers for 50 candidates in under 500 milliseconds.
Stakeholder	Student Project Lead

NFR16	Concurrent Dashboard Loading Throughput
Description	Administrative heatmaps and student search interfaces SHALL load initial views within 1.5 seconds under a concurrent simulated load of 100 active academic users.
Stakeholder	Department Capstone Coordinator, Student Project Lead

3.2.5 Accuracy

NFR17	NLP Entity Extraction Precision & Recall
Description	The zero-shot classification and NER pipeline SHALL achieve at least 85% precision and 80% recall on domain classification across benchmarked academic project statements.
Stakeholder	Student Project Lead, System Administrator

3.2.6 Usability

NFR18	Responsive Multi-Device UI & Accessibility
Description	The web client SHALL conform to WCAG 2.1 Level AA accessibility standards, render responsively across 320px to 4K resolutions, and support dark/light visual modes.
Stakeholder	All Stakeholders

NFR19	Task Completion Efficiency & Intuitive Navigation
Description	A first-time student user SHALL be able to ingest a problem statement, review XAI recommendations, and submit an advisory request within 3 minutes without formal training.
Stakeholder	Student Project Lead

3.2.7 Operational

NFR20	Cross-Platform Browser & OS Compatibility
Description	The web application SHALL function flawlessly across current stable versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari on Windows, macOS, Linux, iOS, and Android.
Stakeholder	All Stakeholders

NFR21	Microservice Modularity & Automated Test Coverage
Description	Core backend services SHALL be decoupled into Dockerized modules, and all algorithmic ranking, FSM transitions, and API endpoints SHALL maintain $\geq 85\%$ automated unit test coverage.
Stakeholder	System Administrator

CHAPTER 4: USE CASE MODELLING

4.1 Use Case Diagram

The high-level interactions between the system actors (Student Project Lead, Faculty Advisor, Industry Mentor, Department Capstone Coordinator, System Administrator, and Institutional Auditor) and the core functional use cases of the UniExpert platform are modeled in Figure 4.1.

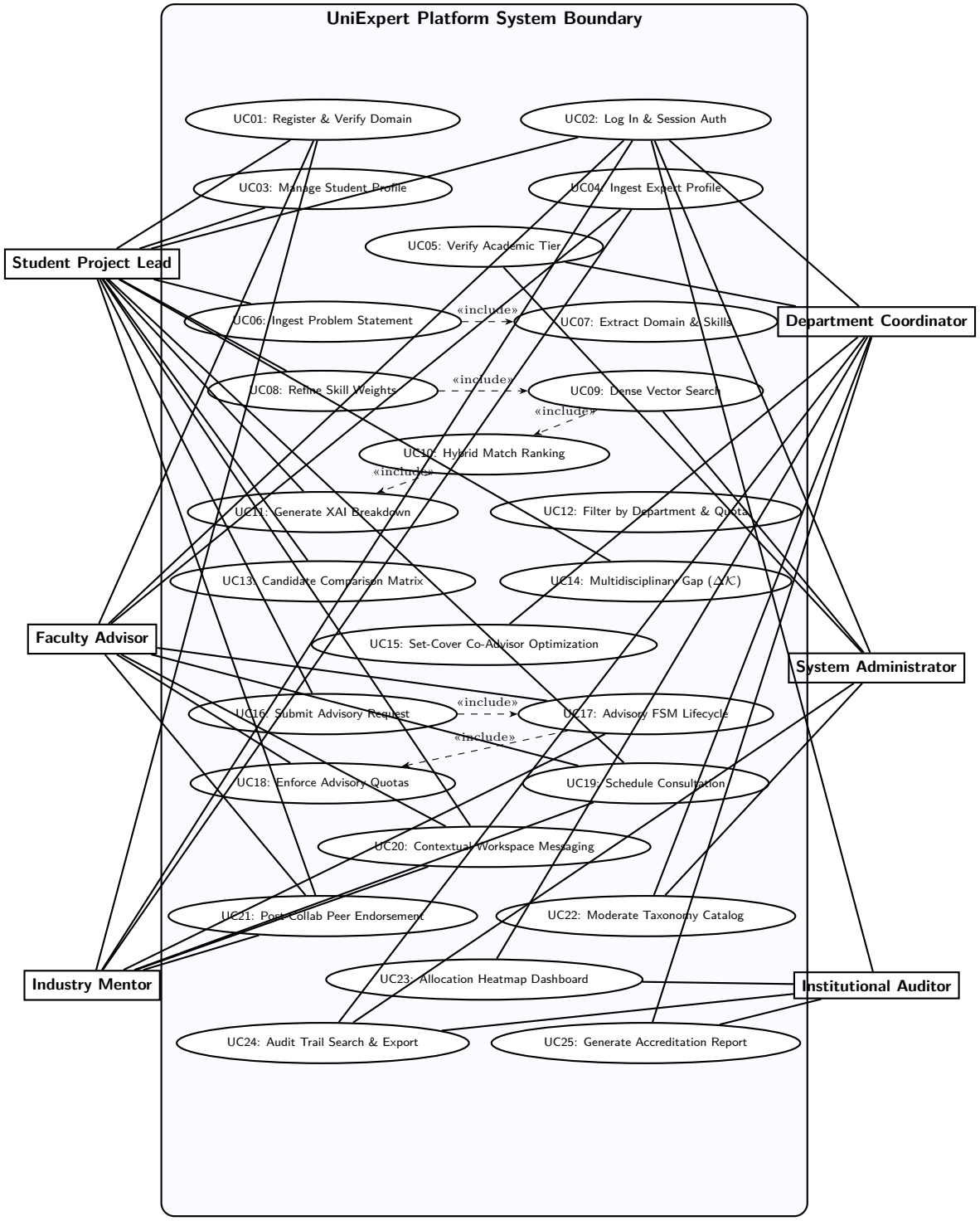


Figure 4.1: System Use Case Diagram for UniExpert Platform

4.2 Use Case Description

The 25 comprehensive use case descriptions below provide formal behavioral contracts specifying goals, preconditions, success and failure end conditions, actor roles, triggers, step-by-step main success scenarios, alternative branches, and non-functional quality requirements.

4.2.1 Use Case Description-01: Register Account & Verify Academic Domain

Use Case	**UC01: Register Account & Verify Academic Domain**
Goal	A new student, faculty expert, or industry mentor creates an authenticated account with verified institutional credentials.
Preconditions	The user possesses a valid university email address (`.edu` or affiliated domain); the registration service is online.
Success End Condition	Account record saved in database; cryptographic verification email dispatched; status `PENDING_VERIFICATION`. Message: *"Account registered successfully. Please verify your institutional email."*
Failed End Condition	No user record created. Message: *"Registration failed: Email address already registered or invalid institutional domain."*
Primary Actors: **Secondary Actors:**	Student Project Lead, Faculty Advisor, Industry Mentor System Administrator
Trigger	User accesses the web portal and clicks "Register Institutional Account".
Description / Main Success Scenario	1. User opens the registration portal and selects their designated institutional role. 2. User enters full legal name, institutional email, department affiliation, and strong password. 3. System validates email format against approved university domain whitelist. 4. System verifies that the email address is not already registered. 5. System computes Argon2id salted password hash and creates a user record with status `PENDING_VERIFICATION`. 6. System generates a cryptographically signed HMAC token (24-hour TTL) and dispatches an activation link. 7. User clicks the activation link in their university inbox. 8. System verifies token authenticity, updates user status to `ACTIVE`, and redirects to role onboarding.
Alternative Flows	**3.a. Email domain is not in university whitelist:** 3.a.1. If registering as Student/Faculty, system rejects registration and prompts for institutional email. 3.a.2. If registering as Industry Mentor, system flags account for manual administrator review. **4.a. Email already exists in system:** 4.a.1. System displays an error notification and offers password reset link. **7.a. Verification token expired (>24 hours):** 7.a.1. System displays expiration notice and offers to generate a fresh verification token.
Quality Requirements	1. Store passwords using salted Argon2id hashes (memory: 64MB, iterations: 3, parallelism: 4). 2. Complete end-to-end registration validation and token generation within 500 milliseconds.

4.2.2 Use Case Description-02: Log In & Session Authentication

Use Case	UC02: Log In & Session Authentication
Goal	An approved, verified user authenticates credentials and establishes a secure, role-scoped session.
Preconditions	User account is verified and active; account is not currently locked out due to excessive failed attempts.
Success End Condition	User session established; short-lived JWT access token issued; secure HTTP-only refresh token set. Message: *"Authentication successful."*
Failed End Condition	Access denied; session rejected. Message: *"Invalid email or password. Please try again."*
Primary Actors: **Secondary Actors:**	All Users (Student, Faculty, Mentor, Coordinator, Admin, Auditor) None
Trigger	User navigates to the login screen and submits credentials.
Description / Main Success Scenario	1. User inputs registered email address and password. 2. System verifies that the account is active and not locked. 3. System verifies submitted password against stored Argon2id hash. 4. System checks if multi-factor authentication (MFA) is mandated for user role. 5. System generates a signed JWT access token containing user UUID, role scopes, and department ID. 6. System issues an HTTP-only, secure, SameSite refresh token cookie (7-day TTL). 7. System logs successful authentication event with client IP and User-Agent in the audit trail. 8. System redirects user to their personalized role dashboard.
Alternative Flows	**2.a. Account locked due to 5 consecutive failed attempts:** 2.a.1. System rejects login attempt and enforces a 15-minute lockout timer. **3.a. Password hash mismatch:** 3.a.1. System increments failed attempt counter in Redis and displays invalid credentials error. **4.a. MFA challenge required (Admin / Coordinator):** 4.a.1. System prompts for 6-digit TOTP code; user submits code; system validates before issuing tokens.
Quality Requirements	1. Access token expiration enforced at 15 minutes; refresh tokens stored in HttpOnly cookies. 2. Enforce 15-minute account lockout after 5 consecutive failed login attempts.

4.2.3 Use Case Description-03: Manage Student Research Profile & Portfolio

Use Case	**UC03: Manage Student Research Profile & Portfolio**
Goal	A student project lead updates team profile details, academic records, CV attachments, and GitHub repositories.
Preconditions	User is signed in with authenticated `STUDENT` role scopes.
Success End Condition	Profile metadata updated in PostgreSQL; searchable profile card refreshed. Message: *"Student profile updated successfully."*
Failed End Condition	Profile update rejected. Message: *"Unable to update profile: Invalid input format or file size exceeded."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Student selects "Edit Profile" on their dashboard.
Description / Main Success Scenario	1. Student opens the profile management dashboard. 2. Student inputs degree program, batch number, student ID, bio, and technical competencies. 3. Student links external portfolio artifacts (GitHub repository URL, LinkedIn profile). 4. Student uploads an optional PDF resume/CV (max 5 MB). 5. System validates file integrity, scans for malicious MIME types, and stores PDF in secure object storage. 6. System persists updated profile entity to PostgreSQL. 7. System updates profile completion index and notifies student of updated profile score.
Alternative Flows	**4.a. Uploaded CV exceeds 5 MB or is non-PDF:** 4.a.1. System rejects file upload and prompts student to attach a valid PDF under 5 MB. **6.a. Database connection timeout during save:** 6.a.1. System rolls back transaction, alerts student, and preserves draft entries in local storage.
Quality Requirements	1. Restrict CV uploads to authenticated students with maximum file size of 5 MB. 2. Update and re-render profile card in under 1 second.

4.2.4 Use Case Description-04: Ingest and Manage Faculty Expert Profile

Use Case	UC04: Ingest and Manage Faculty Expert Profile
Goal	A faculty advisor establishes or synchronizes their research portfolio, publication corpus, and research domains.
Preconditions	User is signed in with `EXPERT` or `MENTOR` role scopes.
Success End Condition	Profile synced; publications parsed and indexed; embeddings queued. Message: *"Expert profile and publications updated."*
Failed End Condition	Synchronization incomplete. Message: *"Publication synchronization failed: External API unreachable."*
Primary Actors: **Secondary Actors:**	Faculty Advisor, Industry Mentor System Administrator
Trigger	Faculty advisor clicks "Sync Google Scholar / ORCID" or manually edits research interests.
Description / Main Success Scenario	1. Faculty advisor navigates to the Research Portfolio settings. 2. Advisor enters their public ORCID iD or Google Scholar profile identifier. 3. System initiates asynchronous background worker to fetch publication metadata, abstracts, and citations. 4. Background worker parses ingested papers, computes h-index, and filters top research topics. 5. System compiles a consolidated academic bio-corpus for the advisor. 6. System updates advisor record in PostgreSQL and triggers vector re-indexing worker. 7. System notifies faculty advisor upon completion of publication ingestion.
Alternative Flows	**3.a. External Scholar API rate limit or network timeout:** 3.a.1. System retries request with exponential backoff (up to 3 retries). 3.a.2. If unsuccessful, system allows manual CSV publication upload. **4.a. No publications detected for entered profile ID:** 4.a.1. System prompts advisor to verify ID or manually input published paper titles.
Quality Requirements	1. Ingest up to 50 publication abstracts within 10 seconds of background job trigger. 2. Preserve existing verified publications if external sync job fails.

4.2.5 Use Case Description-05: Verify Faculty Academic Credentials & Tier Assignment

Use Case	**UC05: Verify Faculty Academic Credentials & Tier Assignment**
Goal	System or administrator validates faculty academic credentials and assigns hierarchical trust badges (Tier 1–3).
Preconditions	Faculty account is registered and email verified; coordinator or admin has review privileges.
Success End Condition	Credential tier updated; verification badge displayed on expert profile. Message: *"Expert verification tier updated to Tier-2."*
Failed End Condition	Verification tier unchanged. Message: *"Verification rejected: Inconclusive academic credentials."*
Primary Actors: **Secondary Actors:**	Department Capstone Coordinator, System Administrator Faculty Advisor
Trigger	Faculty requests credential tier upgrade or coordinator reviews pending departmental faculty list.
Description / Main Success Scenario	1. Coordinator opens the Faculty Verification queue on the administrative console. 2. System displays faculty profile, department, institutional email domain, and synced publications. 3. Coordinator inspects academic credentials and departmental appointment records. 4. Coordinator selects verification tier: Tier-1 (Email Verified), Tier-2 (Scholar Verified), or Tier-3 (Department Head Validated). 5. System persists updated tier in database and logs approval in audit trail. 6. System sends in-app and email notification to faculty advisor confirming tier activation.
Alternative Flows	**3.a. Credentials unverifiable or fraudulent:** 3.a.1. Coordinator rejects tier upgrade and inputs formal justification note. 3.a.2. System marks account as unverified and notifies faculty member to provide proof.
Quality Requirements	1. Restrict credential tier modification strictly to Coordinator and Administrator roles. 2. Record reviewer identity, timestamp, and justification in immutable audit trail.

4.2.6 Use Case Description-06: Ingest Problem Statement & Natural Language Parsing

Use Case	**UC06: Ingest Problem Statement & Natural Language Parsing**
Goal	A student team inputs an unstructured project problem statement and initiates automated NLP processing.
Preconditions	Student lead has an active authenticated session; project proposal form is accessible.
Success End Condition	Problem statement stored; preliminary text tokens parsed and dispatched to entity extractor. Message: *"Problem formulation ingested. Processing NLP extraction."*
Failed End Condition	Ingestion rejected. Message: *"Submission rejected: Problem statement must contain between 50 and 5,000 characters."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Student inputs problem text and clicks "Analyze Proposal".
Description / Main Success Scenario	1. Student navigates to the "Propose Capstone Problem" portal. 2. Student enters project title, target problem description, and envisioned methodology. 3. System validates character count (≥ 50 and $\leq 5,000$ characters). 4. System cleanses input text, strips malicious scripts, and normalizes UTF-8 punctuation. 5. System saves draft project entity in PostgreSQL. 6. System automatically triggers Use Case UC07 (Extract Domain Taxonomy & Skill Entities).
Alternative Flows	**3.a. Text is under 50 characters (insufficient semantic context):** 3.a.1. System warns student: *"Please provide a more detailed problem description to enable accurate AI matching."* **3.b. Text exceeds 5,000 characters:** 3.b.1. System displays character overflow warning and prevents submission until trimmed.
Quality Requirements	1. Sanitize text against cross-site scripting (XSS) and SQL injection payloads. 2. Ingest and save text within 200 milliseconds.

4.2.7 Use Case Description-07: Extract Domain Taxonomy & Prerequisite Skill Entities

Use Case	**UC07: Extract Domain Taxonomy & Prerequisite Skill Entities**
Goal	The AI inference pipeline automatically parses raw project text into macro academic domains and micro technical skill entities.
Preconditions	Problem statement successfully ingested via UC06; NLP microservice is operational.
Success End Condition	Domain tags and skill entity list generated with confidence scores. Message: *"Taxonomy and skill entities extracted."*
Failed End Condition	Extraction fails. Message: *"NLP processing error: Unable to extract skills. Switching to keyword mode."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Automatically triggered following UC06 submission.
Description / Main Success Scenario	1. NLP service receives normalized problem statement text. 2. Service executes zero-shot transformer classification against academic discipline taxonomy. 3. Service identifies top 3 academic domains (e.g., `Embedded Systems`, `Machine Learning`) with confidence ≥ 0.65. 4. Service executes Named Entity Recognition (NER) pipeline to extract technical competencies, libraries, and hardware platforms. 5. Service maps extracted entities to canonical taxonomy terms (e.g., mapping "Tensorflow 2" $\rightarrow$ `TensorFlow`). 6. System structures extracted attributes into JSON payload and transitions to UC08.
Alternative Flows	**3.a. No domain meets 0.65 confidence threshold:** 3.a.1. System selects top 2 nearest domains and flags them as tentative for student confirmation. **4.a. NLP model timeout or container crash:** 4.a.1. System falls back to regex-based keyword matching against academic skill catalog. 4.a.2. System logs fallback event in administrative monitoring console.
Quality Requirements	1. Achieve $\geq$ 85% precision on domain classification across academic benchmark datasets. 2. Complete zero-shot classification and entity extraction within 1.5 seconds.

4.2.8 Use Case Description-08: Refine Extracted Requirements & Skill Weights

Use Case	UC08: Refine Extracted Requirements & Skill Weights
Goal	The student team reviews, adjusts, adds, removes, or re-weights AI-extracted domains and technical skill prerequisites.
Preconditions	Entities extracted via UC07; review card displayed on student screen.
Success End Condition	Validated project requirement vector saved and ready for vector search. Message: *"Requirements confirmed. Searching expert database."*
Failed End Condition	Refinement cancelled. Message: *"Changes discarded."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Student reviews the interactive entity tags generated by the NLP pipeline.
Description / Main Success Scenario	1. Student inspects extracted academic domains and technical skill tags. 2. Student adjusts importance sliders for specific skills (e.g., setting `PyTorch` to High [1.0] and `Docker` to Low [0.3]). 3. Student types to add missing specialized skill tags using autocomplete suggestions. 4. Student clicks "Dismiss" on any irrelevant entities extracted by the NLP model. 5. Student clicks "Confirm Requirements & Find Advisors". 6. System persists finalized requirement weights and triggers vector matchmaking (UC09).
Alternative Flows	**3.a. Student enters custom skill not in system taxonomy:** 3.a.1. System accepts custom entity, flags it for administrator taxonomy review, and indexes it. **5.a. All extracted skills removed by student:** 5.a.1. System prompts student: *"At least one technical skill is required for matching. Please retain or add skills."*
Quality Requirements	1. Autocomplete response time under 100 milliseconds across 2,000+ taxonomy terms. 2. Immediate reactive UI updates when toggling skill sliders.

4.2.9 Use Case Description-09: Execute Dense Semantic Vector Search

Use Case	UC09: Execute Dense Semantic Vector Search
Goal	Retrieve nearest candidate faculty and mentor profiles from high-dimensional vector space using cosine distance.
Preconditions	Project requirements confirmed; Qdrant vector database is reachable and populated.
Success End Condition	Top-50 nearest candidate vectors retrieved with cosine similarity scores. Message: *"Candidate vector retrieval completed."*
Failed End Condition	Search fails. Message: *"Vector search unavailable: Reverting to exact-match database retrieval."*
Primary Actors: **Secondary Actors:**	Student Project Lead System Administrator
Trigger	Student triggers advisor discovery following requirement confirmation.
Description / Main Success Scenario	1. Search service takes confirmed problem text and weighted skill tokens. 2. Service encodes input text into a 384-dimensional dense vector using transformer embedding worker. 3. Service connects to Qdrant vector store and initiates approximate nearest neighbor (ANN) search. 4. HNSW index evaluates cosine distance against indexed faculty research corpora: $$\cos(\vec{q}, \vec{d}_i) = \frac{\vec{q} \cdot \vec{d}_i}{\|\vec{q}\|\|\vec{d}_i\|}$$ 5. Qdrant returns top-50 candidate expert IDs with vector similarity scores ($S_{\text{dense}} \in [0, 1]$). 6. Service passes candidate IDs and scores to Hybrid Match Ranking Engine (UC10).
Alternative Flows	**3.a. Qdrant cluster connection timeout:** 3.a.1. System catches network timeout and initiates fallback to PostgreSQL trigram keyword search. 3.a.2. System delivers fallback results with notification: *"Displaying keyword-matched results."*
Quality Requirements	1. Sub-200ms p95 latency for ANN vector search across 10,000 indexed faculty profiles. 2. Seamless graceful degradation to keyword fallback if vector cluster is offline.

4.2.10 Use Case Description-10: Multi-Criteria Hybrid Match Ranking & Scoring

Use Case	**UC10: Multi-Criteria Hybrid Match Ranking & Scoring**
Goal	Compute composite matching scores combining semantic similarity, skill overlap, publication recency, and availability.
Preconditions	Candidate vector scores returned from UC09; relational profile attributes loaded from PostgreSQL.
Success End Condition	Ranked list of candidate experts generated with composite score $S_{\text{hybrid}} \in [0, 100]$. Message: *"Match ranking completed."*
Failed End Condition	Scoring failed. Message: *"Unable to compute rankings; please retry."*
Primary Actors: **Secondary Actors:**	Student Project Lead Faculty Advisor
Trigger	Candidate vector retrieval completes.
Description / Main Success Scenario	1. Ranking engine fetches full metadata for top-50 candidate IDs from PostgreSQL. 2. Engine calculates normalized skill overlap ratio: $$S_{\text{skill}} = \frac{\lvert\mathcal{K}_{\text{req}} \cap \mathcal{K}_{\text{expert}}\rvert}{\lvert\mathcal{K}_{\text{req}}\rvert}$$ 3. Engine calculates publication recency and domain citation weight (S_{pub}). 4. Engine applies verification tier multiplier ($S_{\text{tier}} \in \{1.0, 1.15, 1.25\}$). 5. Engine evaluates active advisory quota capacity multiplier (S_{avail}). 6. Engine computes overall hybrid composite score: $$S_{\text{hybrid}} = (\omega_1 S_{\text{dense}} + \omega_2 S_{\text{skill}} + \omega_3 S_{\text{pub}} + \omega_4 S_{\text{tier}}) \times S_{\text{avail}}$$ 7. Engine sorts candidates in descending score order and delivers results to client.
Alternative Flows	**5.a. Advisor has 0 remaining supervisory slots ($S_{\text{avail}} = 0$ for primary supervision):** 5.a.1. System adjusts profile status to "Quota Full" and flags candidate for co-advisory only.
Quality Requirements	1. Calculate composite scores and rank 50 candidates in under 300 milliseconds. 2. Composite score formulation normalized strictly between 0 and 100.

4.2.11 Use Case Description-11: Generate Explainable AI (XAI) Recommendation Breakdown

Use Case	**UC11: Generate Explainable AI (XAI) Recommendation Breakdown**
Goal	Provide transparent, human-interpretable justifications for each recommended candidate expert.
Preconditions	Candidate expert list ranked via UC10; student selects a candidate card.
Success End Condition	Detailed XAI justification card and percentage attribution breakdown rendered. Message: *"Match justification loaded."*
Failed End Condition	Justification unavailable. Message: *"Detailed explanation could not be generated."*
Primary Actors: **Secondary Actors:**	Student Project Lead Faculty Advisor, Department Coordinator
Trigger	Student clicks "Why this match?" on a candidate expert profile card.
Description / Main Success Scenario	1. Student clicks "Why this match?" on Dr. Farhana Rahman's profile card. 2. System retrieves score component vectors computed during UC10. 3. System formats component percentage contributions into an interactive breakdown modal: • 45% Semantic Vector Similarity (highlighting 3 matched publication abstracts). • 30% Prerequisite Skill Overlap (displaying matched skills in green). • 15% Verified Academic Credential Tier (Tier-2 Scholar Verified). • 10% Supervisory Bandwidth (2 active slots available). 4. System renders a radar chart comparing student project requirements against expert proficiencies. 5. Student inspects justification and gains confidence in candidate suitability.
Alternative Flows	**3.a. Match is driven almost exclusively by keyword overlap rather than semantic papers:** 3.a.1. System displays an advisory hint: *"Strong practical toolchain overlap; limited recent publication history in this exact domain."*
Quality Requirements	1. Render XAI visual drawer and radar chart within 200 milliseconds of user click. 2. All percentage contributions must sum precisely to 100%.

4.2.12 Use Case Description-12: Filter Candidates by Department, Quota & Availability

Use Case	**UC12: Filter Candidates by Department, Quota & Availability**
Goal	Allow students to dynamically refine candidate results using faceted academic criteria.
Preconditions	Candidate recommendation results active on the search screen.
Success End Condition	Filtered candidate list updated in real time. Message: *"Filters applied."*
Failed End Condition	No candidates match filter criteria. Message: *"No experts match the selected filters. Try broadening your criteria."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Student selects filter checkboxes on the search sidebar.
Description / Main Success Scenario	1. Student opens the faceted search filter panel. 2. Student selects filter criteria: Department = "Software Engineering", Quota Status = "Available Now", Credential Tier = "Tier-2+". 3. Client dispatches filter parameters to backend API. 4. System applies filter predicates against cached recommendation candidate set. 5. System updates UI with filtered candidate list and displays matching count.
Alternative Flows	**4.a. Filter results in 0 matches:** 4.a.1. System displays "Zero Matches Found" empty state card. 4.a.2. System provides a 1-click "Reset All Filters" action to restore recommendations.
Quality Requirements	1. Apply faceted filters and update client view in under 150 milliseconds. 2. Maintain active filter state across browser tab refreshes.

4.2.13 Use Case Description-13: Perform Side-by-Side Expert Comparison

Use Case	UC13: Perform Side-by-Side Expert Comparison
Goal	Compare up to three candidate advisors across standardized academic and availability metrics.
Preconditions	Student has identified multiple candidate experts in search results.
Success End Condition	Side-by-side comparison matrix rendered with differential metric highlights. Message: *"Comparison matrix ready."*
Failed End Condition	Comparison failed. Message: *"Unable to load comparison: Select between 2 and 3 candidates."*
Primary Actors: **Secondary Actors:**	Student Project Lead None
Trigger	Student checks "Compare" on two or three candidate cards and clicks "Compare Selected".
Description / Main Success Scenario	1. Student selects two or three candidate experts from search results. 2. Student clicks "Compare Selected Experts". 3. System loads full profile attributes for chosen experts from database. 4. System renders a tabular comparative matrix aligning: • Overall Match Score and Domain Similarity. • Matched vs. Missing Technical Skills. • Total Citations, h-index, and Recent Publication Count (last 3 years). • Available Supervisory Capacity and Office Hour Availability. • Average Post-Collaboration Student Rating. 5. Student analyzes relative strengths and selects preferred primary supervisor.
Alternative Flows	**1.a. Student attempts to select more than 3 candidates:** 1.a.1. System displays notification: *"You can compare a maximum of 3 candidates simultaneously."*
Quality Requirements	1. Render comparison matrix in under 300 milliseconds. 2. Highlight differential metrics clearly with color-coded comparison badges.

4.2.14 Use Case Description-14: Analyze Multidisciplinary Team Skill Gap ($\Delta\mathcal{K}$)

Use Case	**UC14: Analyze Multidisciplinary Team Skill Gap ($\Delta\mathcal{K}$)**
Goal	Detect unaddressed prerequisite technical skills in a student project team and evaluate project viability.
Preconditions	Project proposal requirements defined; student team member profiles linked to project.
Success End Condition	Skill gap report generated showing addressed vs. missing prerequisite competencies. Message: *"Skill gap analysis complete."*
Failed End Condition	Analysis failed. Message: *"Unable to compute skill gaps: Team roster missing."*
Primary Actors: **Secondary Actors:**	Student Project Lead, Department Capstone Coordinator None
Trigger	Student lead or coordinator clicks "Analyze Team Skill Gaps".
Description / Main Success Scenario	1. System retrieves project prerequisite skill set $\mathcal{K}_{\text{req}}$ from project record. 2. System queries profiles of all confirmed team members to compute collective skill union: $$\mathcal{K}_{\text{team}} = \bigcup_{m \in \text{Members}} \mathcal{K}_m$$ 3. System calculates the unaddressed deficit set: $\Delta\mathcal{K} = \mathcal{K}_{\text{req}} \setminus \mathcal{K}_{\text{team}}$. 4. System computes team readiness score: $$R_{\text{team}} = 1 - \frac{\lvert\Delta\mathcal{K}\rvert}{\lvert\mathcal{K}_{\text{req}}\rvert}$$ 5. System displays visual breakdown highlighting covered skills (green) and critical deficits (red). 6. System displays prompt: *"Would you like to discover co-advisors or peers to resolve missing skills?"*
Alternative Flows	**3.a. Deficit set is empty ($\Delta\mathcal{K} = \emptyset$):** 3.a.1. System displays "100% Team Skill Coverage" green banner and bypasses co-advisor prompt.
Quality Requirements	1. Compute skill set union and set difference in under 50 milliseconds. 2. Accurately flag interdisciplinary skill deficits across department boundaries.

4.2.15 Use Case Description-15: Run Set-Cover Optimization for Co-Advisor Selection

Use Case	**UC15: Run Set-Cover Optimization for Co-Advisor Selection**
Goal	Formulate and solve a combinatorial set-cover problem to recommend the minimal set of co-advisors to cover team gaps.
Preconditions	Skill deficit set $\Delta\mathcal{K} \neq \emptyset$ identified via UC14; faculty pool indexed.
Success End Condition	Optimal co-advisor recommendations presented with simulated post-addition coverage. Message: *"Optimal co-advisor candidates identified."*
Failed End Condition	Optimization failed. Message: *"No co-advisors found covering the specified skill gaps."*
Primary Actors: **Secondary Actors:**	Student Project Lead, Department Capstone Coordinator Faculty Advisor
Trigger	User selects "Find Co-Advisors to Bridge Skill Gaps".
Description / Main Success Scenario	1. System loads unaddressed skill set $\Delta\mathcal{K}$. 2. System queries candidate faculty members outside the primary supervisor's department. 3. System executes a greedy approximation algorithm for weighted set-cover to select candidates that maximize: $$\arg\max_{e \in \text{Experts}} \frac{\lvert\mathcal{K}_e \cap \Delta\mathcal{K}\rvert}{\text{Cost}(e)}$$ 4. System ranks candidate co-advisors based on marginal skill contribution and quota availability. 5. System presents top co-advisor recommendations with a simulation card: *"Adding Dr. Tanvir Ahmed (Bioinformatics) increases team coverage from 40% to 92%."* 6. Student lead clicks "Propose Co-Advisory Collaboration".
Alternative Flows	**3.a. No single faculty covers all deficits:** 3.a.1. System recommends a paired combination of one co-advisor and one external industry mentor.
Quality Requirements	1. Execute greedy set-cover algorithm in under 200 milliseconds. 2. Guarantee that recommended set strictly reduces cardinality of $\Delta\mathcal{K}$.

4.2.16 Use Case Description-16: Submit Collaboration Advisory Request

Use Case	UC16: Submit Collaboration Advisory Request
Goal	A student project team submits a formal, structured capstone advisory proposal to a selected faculty expert.
Preconditions	Student lead has active project draft; selected faculty has available quota (> 0 active slots).
Success End Condition	Advisory proposal record created; FSM transitions to `SUBMITTED`; notification sent. Message: *"Advisory request submitted successfully."*
Failed End Condition	Submission blocked. Message: *"Unable to submit request: Faculty quota full or pending request already exists."*
Primary Actors: **Secondary Actors:**	Student Project Lead Faculty Advisor
Trigger	Student clicks "Send Advisory Proposal" on expert profile page.
Description / Main Success Scenario	1. Student opens the advisory proposal modal. 2. Student reviews pre-populated project synopsis and AI alignment score. 3. Student enters structured proposal items: envisioned milestones, expected meeting frequency, and personal motivation statement. 4. Student selects advisory scope: Primary Supervisor or Co-Advisor. 5. System validates that the student team has no other pending requests to the same professor. 6. System verifies that the target faculty member's available quota is ≥ 1. 7. System creates an `AdvisoryRequest` record in PostgreSQL with status `SUBMITTED`. 8. System starts a 7-day auto-response countdown timer. 9. System delivers dashboard notification and email digest to the faculty advisor.
Alternative Flows	**6.a. Faculty quota reached while student was editing form:** 6.a.1. System blocks submission: *"Professor's quota was filled by another group. Proposal cannot be submitted."* 6.a.2. System prompts student to select an alternative recommended advisor.
Quality Requirements	1. Enforce atomic database transaction to prevent race conditions on quota capacity. 2. Dispatch confirmation notification within 5 seconds of submission.

4.2.17 Use Case Description-17: Manage Advisory Request Lifecycle (Accept / Decline / Negotiate)

Use Case	**UC17: Manage Advisory Request Lifecycle (Accept / Decline / Negotiate)**
Goal	A faculty advisor evaluates an incoming student collaboration proposal and executes an FSM state transition.
Preconditions	Faculty advisor is signed in; advisory proposal in `SUBMITTED` or `UNDER_REVIEW` state.
Success End Condition	FSM state updated to `ACCEPTED`, `DECLINED`, or `NEGOTIATION`. Message: *"Advisory request status updated."*
Failed End Condition	State transition rejected. Message: *"Invalid state transition or proposal expired."*
Primary Actors: **Secondary Actors:**	Faculty Advisor Student Project Lead, Department Capstone Coordinator
Trigger	Faculty advisor opens a pending request from their "Incoming Solicitations" queue.
Description / Main Success Scenario	1. Faculty advisor opens pending proposal in the advisory portal. 2. System updates proposal state to `UNDER_REVIEW`. 3. Advisor reviews student project statement, proposed milestones, and XAI match score. 4. Advisor selects action: • **Accept:** FSM transitions to `ACCEPTED`; advisor quota decremented; active workspace created. • **Negotiate:** Advisor enters milestone modification feedback; FSM transitions to `NEGOTIATION`. • **Decline:** Advisor inputs polite declination reason; FSM transitions to `DECLINED`. 5. System persists state transition with timestamp and user ID in audit trail. 6. System triggers real-time notification to student project team.
Alternative Flows	**4.a. 7-day response timer expires without advisor action:** 4.a.1. System background cron job automatically transitions status to `EXPIRED`. 4.a.2. System releases student team from waiting lock and suggests next-best ranked advisors.
Quality Requirements	1. Enforce strict mathematical state transition rules with no illegal skips. 2. Log every transition and feedback comment in immutable audit ledger.

4.2.18 Use Case Description-18: Enforce Faculty Advisory Quotas & Blackout Windows

Use Case	**UC18: Enforce Faculty Advisory Quotas & Blackout Windows**
Goal	Maintain balanced faculty supervision loads and prevent inbox burnout by enforcing hard quota caps and blackout dates.
Preconditions	Faculty member has active profile; department capstone quota policies defined.
Success End Condition	Quota settings updated; availability status dynamically toggled. Message: *"Supervisory quota settings saved."*
Failed End Condition	Settings rejected. Message: *"Invalid quota value: Must be between 1 and 10 groups."*
Primary Actors: **Secondary Actors:**	Faculty Advisor, Department Capstone Coordinator System Administrator
Trigger	Faculty advisor configures settings or an advisory proposal is accepted.
Description / Main Success Scenario	1. Faculty advisor accesses the "Advisory Capacity & Quotas" settings tab. 2. Advisor sets maximum active capstone supervision groups (e.g., 4 groups). 3. Advisor designates blackout calendar windows (e.g., sabbatical dates or conference travel). 4. System validates settings against departmental statutory policy limits. 5. System recalculates current active count: ActiveGroups = Count(AcceptedProposals). 6. If ActiveGroups $\geq$ MaxQuota, system automatically transitions status to `QUOTA_REACHED`. 7. System updates expert availability index in vector store to depress primary supervisor ranking.
Alternative Flows	**4.a. Coordinator enacts departmental quota override:** 4.a.1. Coordinator elevates faculty cap by +1 group due to special research necessity. 4.a.2. System records coordinator override rationale in audit log.
Quality Requirements	1. Real-time quota enforcement with atomic database locking on acceptance. 2. Asynchronous status update to vector search index within 2 seconds.

4.2.19 Use Case Description-19: Schedule Mentorship Consultation Session

Use Case	UC19: Schedule Mentorship Consultation Session
Goal	An approved student team schedules an office-hour consultation session with their assigned advisor.
Preconditions	Active accepted advisory relationship exists; advisor has published open consultation slots.
Success End Condition	Consultation booked; calendar invite generated; status `SCHEDULED`. Message: *"Consultation booked successfully."*
Failed End Condition	Booking rejected. Message: *"Slot no longer available or agenda missing."*
Primary Actors: **Secondary Actors:**	Student Project Lead Faculty Advisor, Industry Mentor
Trigger	Student opens the "Schedule Consultation" view in active project workspace.
Description / Main Success Scenario	1. Student opens the advisor's consultation calendar. 2. System displays open 30-minute office-hour slots for the upcoming two weeks. 3. Student selects an open slot (e.g., Thursday 3:00 PM – 3:30 PM). 4. Student enters mandatory session agenda and attaches discussion documents. 5. System verifies that the slot has not been concurrently reserved. 6. System creates consultation record in database and marks slot as `BOOKED`. 7. System generates an iCalendar (.ics) invite and sends email confirmation to both parties. 8. System logs scheduled session in the project workspace milestone tracker.
Alternative Flows	**5.a. Concurrent booking conflict (another student books slot simultaneously):** 5.a.1. System detects database slot lock collision. 5.a.2. System alerts student: *"Slot was just booked by another team. Please choose another time."*
Quality Requirements	1. Prevent double-booking using strict PostgreSQL row-level locks on calendar slots. 2. Dispatch calendar invitations and notifications within 3 seconds of booking.

4.2.20 Use Case Description-20: Exchange Contextual Messages & Project Milestones

Use Case	UC20: Exchange Contextual Messages & Project Milestones
Goal	Students and advisors communicate, share milestone reports, and log technical feedback inside project workspace.
Preconditions	User belongs to an active `ACCEPTED` project workspace.
Success End Condition	Message/file persisted; conversation stream updated in real time. Message: *"Message posted."*
Failed End Condition	Transmission failed. Message: *"Failed to send message: File type not permitted or network offline."*
Primary Actors: **Secondary Actors:**	Student Project Lead, Faculty Advisor, Industry Mentor None
Trigger	User inputs text or attaches a document in the workspace discussion channel.
Description / Main Success Scenario	1. User opens the workspace discussion channel for their capstone project. 2. User enters message text or selects a milestone artifact (e.g., `Architecture_v1.pdf`). 3. System validates file attachment format (PDF, DOCX, ZIP; max 25 MB). 4. System scans file for malicious payloads and stores it in encrypted object storage. 5. System saves message record to database linked to the project and author ID. 6. System broadcasts new message via WebSocket connection to active workspace participants. 7. System updates unread message badge count for offline team members.
Alternative Flows	**3.a. Attachment exceeds 25 MB limit:** 3.a.1. System rejects file and suggests providing an external repository link (e.g., GitHub/Drive).
Quality Requirements	1. WebSocket message delivery latency under 100 milliseconds for connected clients. 2. All messages encrypted at rest using AES-256-GCM.

4.2.21 Use Case Description-21: Submit Post-Collaboration Evaluation & Endorsement

Use Case	UC21: Submit Post-Collaboration Evaluation & Endorsement
Goal	Record mutual structured evaluations, quantitative ratings, and verified skill endorsements upon capstone completion.
Preconditions	Capstone project marked as completed by coordinator; advisory relationship closed.
Success End Condition	Mutual evaluation saved; reputation scores updated; skill attestations verified. Message: *"Evaluation submitted successfully."*
Failed End Condition	Submission rejected. Message: *"Evaluation submission incomplete: Required rubric items missing."*
Primary Actors: **Secondary Actors:**	Student Project Lead, Faculty Advisor, Industry Mentor Department Capstone Coordinator
Trigger	System prompts user for post-project review upon semester conclusion.
Description / Main Success Scenario	1. User accesses the "Project Completion Review" portal. 2. User completes a structured quantitative rubric (responsiveness, technical depth, milestone adherence). 3. User inputs qualitative feedback comments. 4. Advisor endorses specific skills demonstrated by students during the project. 5. System validates that all mandatory evaluation fields are completed. 6. System records evaluation in PostgreSQL with status `COMMITTED`. 7. System updates the recipient's aggregate reputation score and attaches skill badges.
Alternative Flows	**3.a. User submits derogatory or non-academic commentary:** 3.a.1. Automated content moderation filter flags text and routes review to Department Coordinator.
Quality Requirements	1. Maintain confidentiality of student reviews until both parties submit or deadline passes. 2. Update aggregated reputation metric immediately upon submission commit.

4.2.22 Use Case Description-22: Moderate Content & Re-Index Ontology Taxonomy

Use Case	**UC22: Moderate Content & Re-Index Ontology Taxonomy**
Goal	Administrator updates academic skill taxonomy, moderates custom tags, and triggers vector re-indexing.
Preconditions	Administrator is signed in with `ADMIN` superuser privileges.
Success End Condition	Taxonomy nodes updated; vector index refresh queued. Message: *"Taxonomy updated and re-indexing initiated."*
Failed End Condition	Operation failed. Message: *"Unable to update taxonomy: Cyclic hierarchy detected."*
Primary Actors: **Secondary Actors:**	System Administrator, Department Capstone Coordinator None
Trigger	Administrator opens the Taxonomy & Content Moderation Console.
Description / Main Success Scenario	1. Administrator opens the Academic Taxonomy Tree view. 2. Administrator adds new technical skill entities (e.g., `LangChain`, `TinyML`) under parent domain nodes. 3. Administrator reviews and approves or merges user-submitted custom skill tags. 4. System verifies that the taxonomy remains an acyclic directed graph (DAG). 5. Administrator clicks "Re-Index Academic Embeddings". 6. System triggers background Celery worker to regenerate taxonomy vectors in Qdrant. 7. System logs taxonomy modification event in administrative audit ledger.
Alternative Flows	**4.a. Administrator attempts to create circular relationship:** 4.a.1. System rejects operation: *"Circular dependency detected between domain nodes."*
Quality Requirements	1. Validate taxonomy DAG integrity within 100 milliseconds. 2. Execute vector collection re-indexing asynchronously without interrupting search queries.

4.2.23 Use Case Description-23: Monitor Departmental Supervisory Allocation Dashboard

Use Case	UC23: Monitor Departmental Supervisory Allocation Dashboard
Goal	Coordinator monitors real-time faculty supervisory distribution, unserved student teams, and quota health.
Preconditions	Coordinator or auditor is signed in with administrative view permissions.
Success End Condition	Supervisory heatmap and unallocated student queues rendered. Message: *"Departmental dashboard loaded."*
Failed End Condition	Dashboard failed to load. Message: *"Analytics data unavailable."*
Primary Actors: **Secondary Actors:**	Department Capstone Coordinator, Institutional Auditor None
Trigger	Coordinator navigates to "Departmental Allocation Console".
Description / Main Success Scenario	1. Coordinator opens the Department Allocation Dashboard. 2. System aggregates real-time data across all active capstone teams and faculty rosters. 3. System renders an interactive allocation heatmap showing active student load per faculty member. 4. System highlights faculty members who are overloaded or have reached maximum quota. 5. System displays a dedicated list of "Unallocated Student Teams" who have no confirmed advisor. 6. Coordinator filters data by academic semester, program, or specialization track.
Alternative Flows	**5.a. Coordinator identifies an unallocated team whose requests all expired:** 5.a.1. Coordinator uses administrative action to assign an available co-advisor directly.
Quality Requirements	1. Load aggregate dashboard views for 200+ faculty and 500+ teams in under 1.5 seconds. 2. Refresh data automatically every 60 seconds or on manual refresh click.

4.2.24 Use Case Description-24: Search and Export Institutional Compliance Audit Trail

Use Case	**UC24: Search and Export Institutional Compliance Audit Trail**
Goal	Trace system activities, review cryptographic transaction logs, and export verifiable audit records.
Preconditions	Administrator or institutional auditor has verified administrative credentials.
Success End Condition	Filtered audit records displayed; encrypted CSV/JSON audit file exported. Message: *"Audit trail exported successfully."*
Failed End Condition	Search or export failed. Message: *"Audit search query failed: Invalid parameters."*
Primary Actors: **Secondary Actors:**	System Administrator, Institutional Auditor, Department Coordinator None
Trigger	Auditor accesses the "Audit Trail & Governance" portal.
Description / Main Success Scenario	1. Auditor navigates to the Audit Trail interface. 2. Auditor defines search criteria: Date Range, Actor Role, Event Type (e.g., `FSM_STATE_TRANSITION`), and User UUID. 3. System queries the append-only audit database table. 4. System validates cryptographic hash chaining across selected log records. 5. System renders matching audit events with timestamp, user ID, IP address, and payload diff. 6. Auditor clicks "Export Signed Audit CSV". 7. System compiles audit dataset, attaches SHA-256 integrity signature, and serves download.
Alternative Flows	**4.a. Cryptographic log chain mismatch detected (tampering attempt):** 4.a.1. System displays high-severity security alert and isolates affected record range. 4.a.2. System dispatches immediate emergency security alert to System Administrators.
Quality Requirements	1. Enforce append-only integrity; reject any UPDATE or DELETE operations on audit log tables. 2. Export 10,000 audit log records to CSV in under 2 seconds.

4.2.25 Use Case Description-25: Generate Departmental Capstone Analytics & Accreditation Report

Use Case	UC25: Generate Departmental Capstone Analytics & Accreditation Report
Goal	Generate certified PDF and CSV institutional summary reports for accreditation boards (e.g., BAETE/IEB).
Preconditions	Authorized coordinator or auditor session active; semester capstone records synchronized.
Success End Condition	Certified institutional report generated and downloaded. Message: *"Accreditation report ready for download."*
Failed End Condition	Generation failed. Message: *"Unable to compile report: Incomplete data for selected semester."*
Primary Actors: **Secondary Actors:**	Department Capstone Coordinator, Institutional Auditor System Administrator
Trigger	Coordinator navigates to "Reports & Accreditation" and clicks "Generate Report".
Description / Main Success Scenario	1. Coordinator opens the Reports console. 2. Coordinator selects report type: "Semester Supervisory Allocation Summary" or "Interdisciplinary Research Velocity". 3. Coordinator selects academic term (e.g., "Fall 2026") and target department. 4. Coordinator selects output format: PDF (formal executive report) or CSV (raw data). 5. System aggregates metrics: total projects, supervisor-to-student ratios, multidisciplinary co-advisory percentage, and milestone adherence rates. 6. System compiles the report document with institutional header, summary tables, and charts. 7. System generates downloadable file and logs report generation in audit trail.
Alternative Flows	**5.a. No finalized projects exist for selected term:** 5.a.1. System displays notice: *"No completed records found for the selected semester."*
Quality Requirements	1. Render and compile multi-page PDF accreditation report within 3 seconds. 2. Exclude private student contact details from aggregate institutional reports.

CHAPTER 5: SYSTEM DESIGN

5.1 Data Flow Diagram

The functional data architecture and information boundaries of the UniExpert platform are formalized through hierarchical Data Flow Diagrams (DFDs).

5.1.1 Data Flow Diagram Level-0

The Level-0 Context Diagram (Figure 5.1) establishes the foundational boundary between the external actors and the centralized UniExpert system.

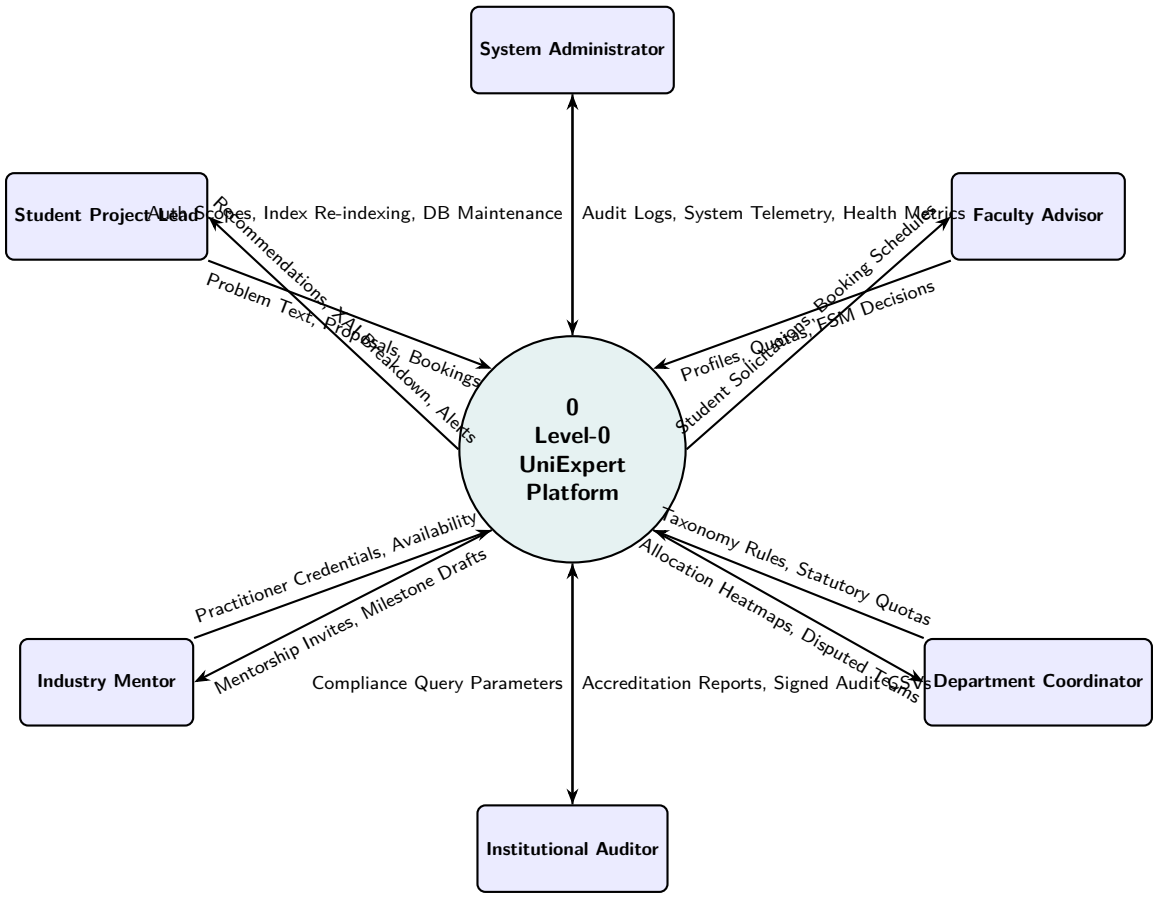


Figure 5.1: Data Flow Diagram Level-0 (Context Diagram)

5.1.2 Data Flow Diagram Level-1

The Level-1 Data Flow Diagram (Figure 5.2) decomposes the centralized system into six functional microservice processes and six persistent data repositories.

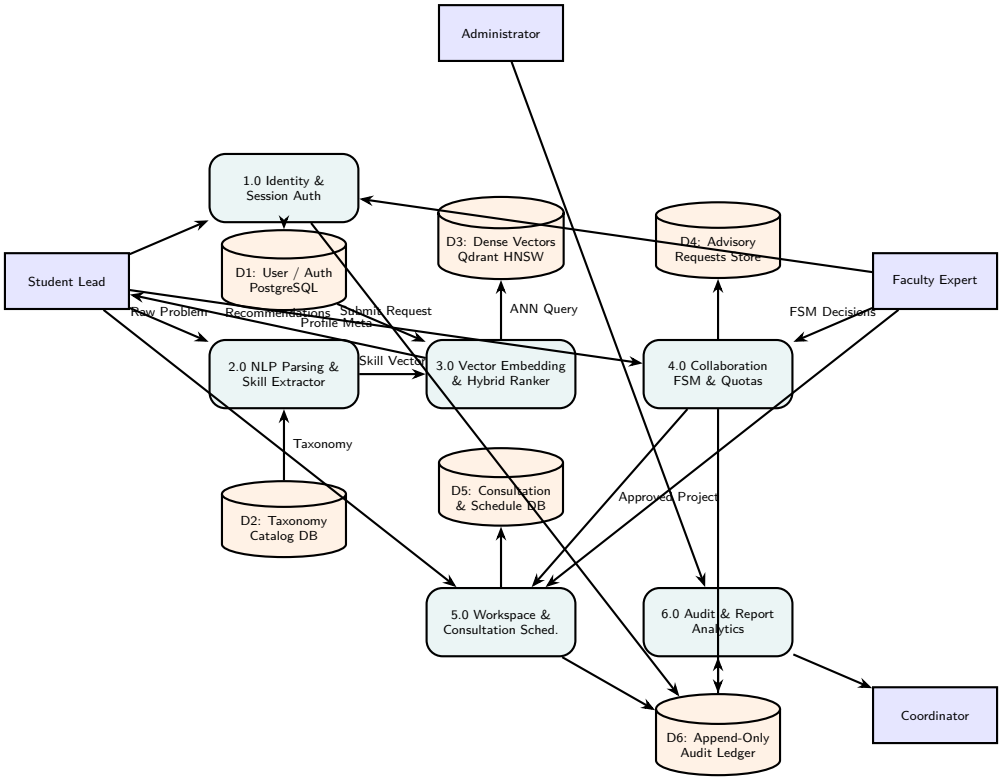


Figure 5.2: Data Flow Diagram Level-1 (Functional Decomposition)

5.2 Sequence Diagram

The dynamic message exchanges, component activations, and lifecycle lifelines across system boundaries are detailed in the six sequence diagrams below.

5.2.1 Sequence Diagram-01: Problem Statement Ingestion & AI Expert Recommendation

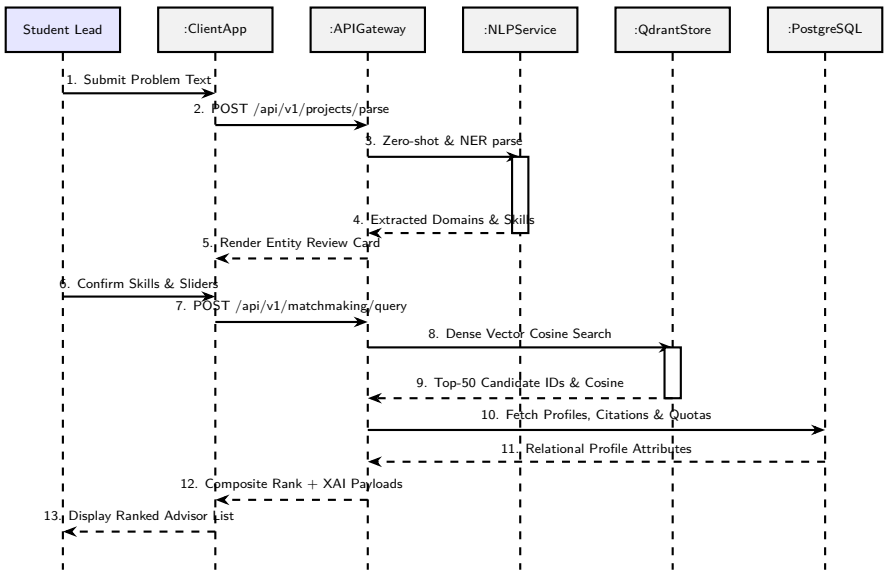


Figure 5.3: Sequence Diagram for Problem Ingestion & AI Recommendation

5.2.2 Sequence Diagram-02: Advisory Collaboration Request & FSM Lifecycle Negotiation

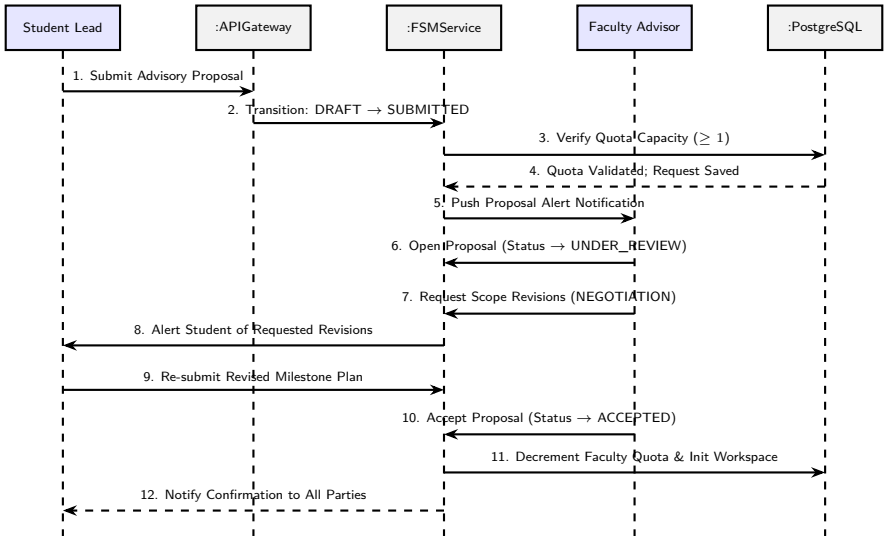


Figure 5.4: Sequence Diagram for Advisory Request FSM Lifecycle

5.2.3 Sequence Diagram-03: Expert Profile Ingestion & Dense Vector Indexing

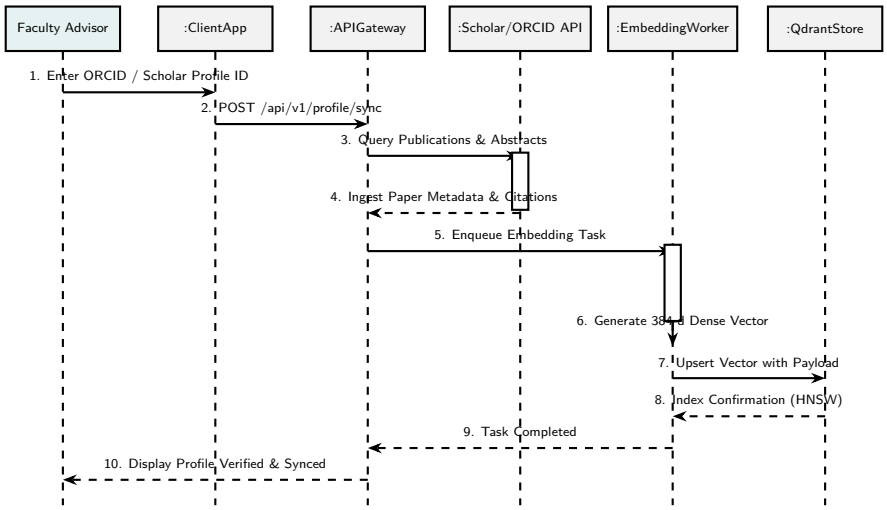


Figure 5.5: Sequence Diagram for Expert Profile Ingestion & Vector Indexing

5.2.4 Sequence Diagram-04: Multidisciplinary Team Skill Gap Analysis & Co-Advisor Discovery

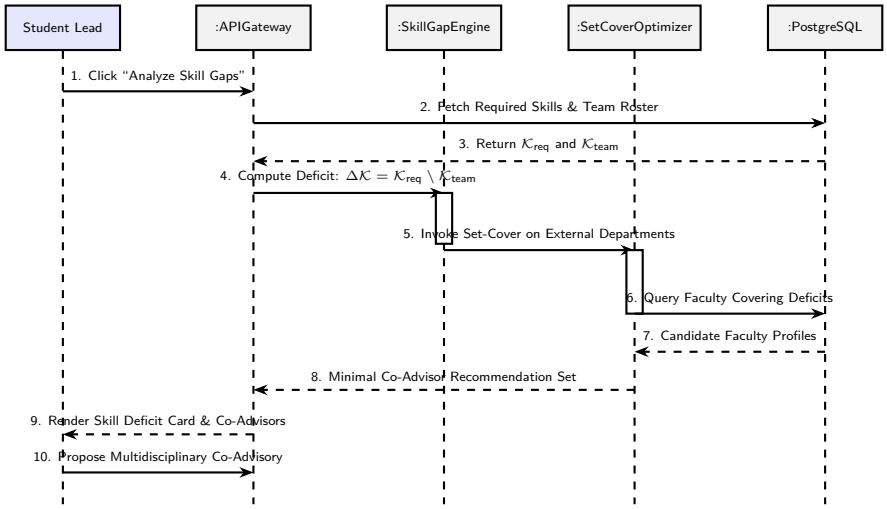


Figure 5.6: Sequence Diagram for Multidisciplinary Skill Gap Optimization

5.2.5 Sequence Diagram-05: Mentorship Consultation Scheduling & Availability Check

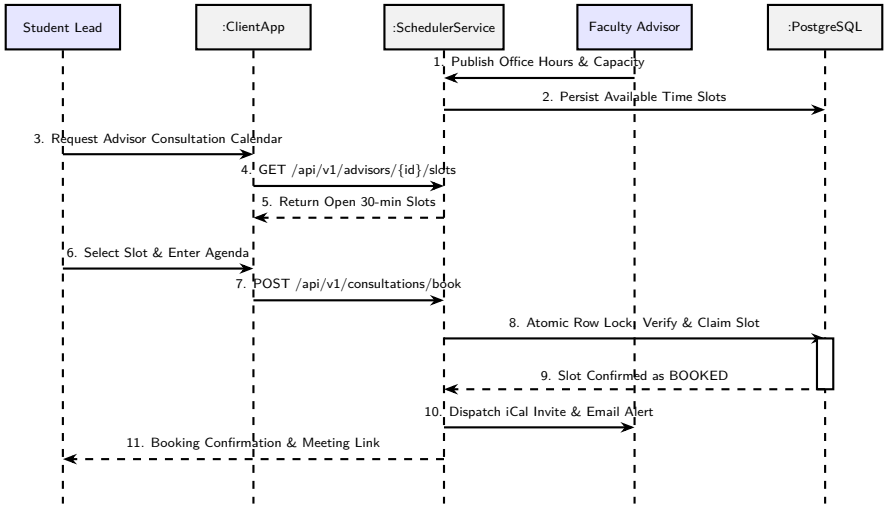


Figure 5.7: Sequence Diagram for Consultation Scheduling

5.2.6 Sequence Diagram-06: Institutional Compliance Audit Search & Export

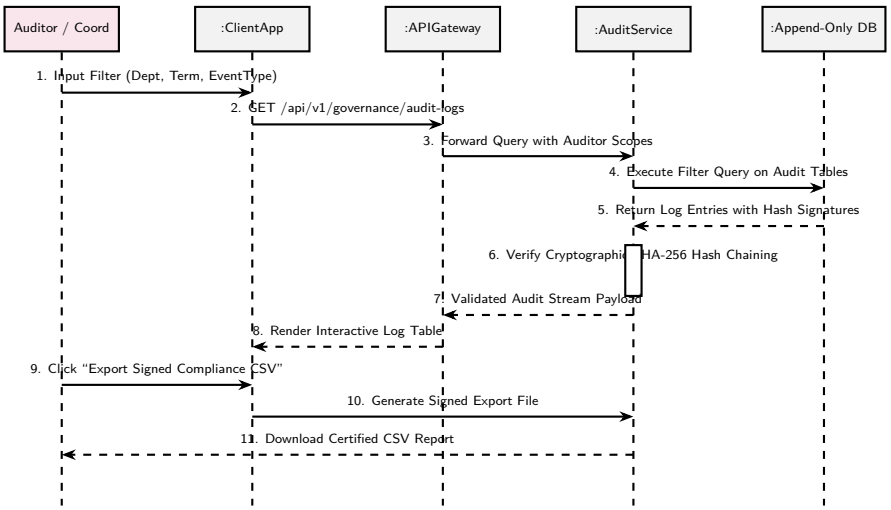


Figure 5.8: Sequence Diagram for Institutional Compliance Audit Search

5.3 State Diagram

The discrete behavioral states, transition triggers, guard conditions, and exit actions governing entities in the UniExpert platform are captured in the six state diagrams below.

5.3.1 State Diagram-01: Advisory Request FSM Lifecycle

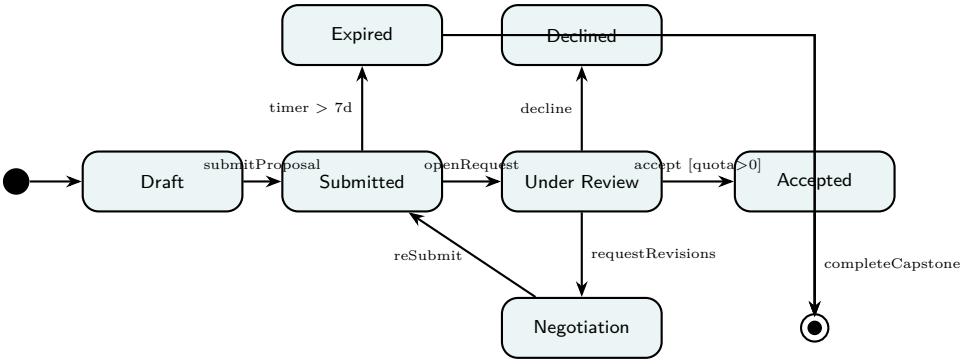


Figure 5.9: State Diagram for Advisory Request FSM Lifecycle

5.3.2 State Diagram-02: User Account Status

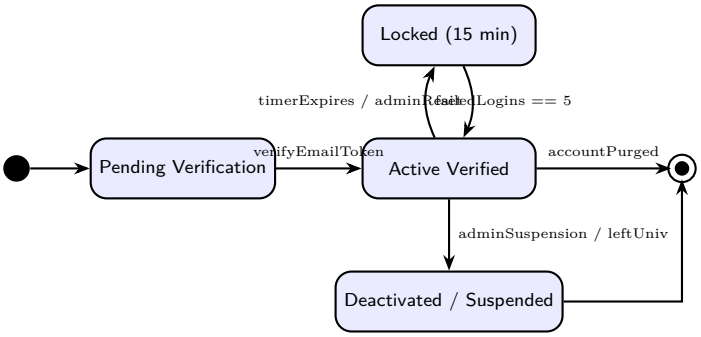


Figure 5.10: State Diagram for User Account Status

5.3.3 State Diagram-03: Faculty Advisory Quota & Availability State

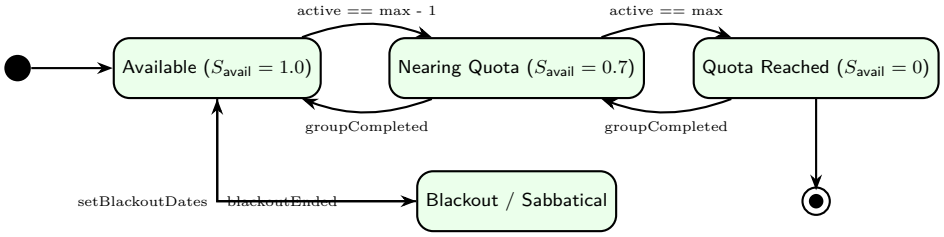


Figure 5.11: State Diagram for Faculty Advisory Quota Availability

5.3.4 State Diagram-04: Capstone Project Stage Lifecycle

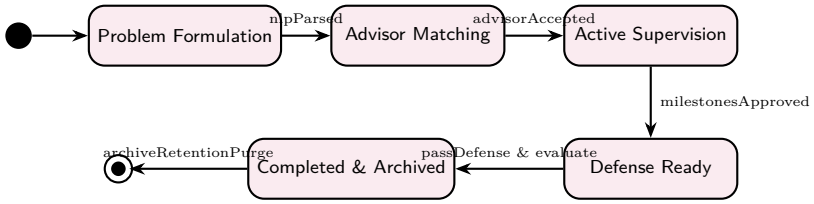


Figure 5.12: State Diagram for Capstone Project Stage Lifecycle

5.3.5 State Diagram-05: AI Inference & Matchmaking Pipeline Status

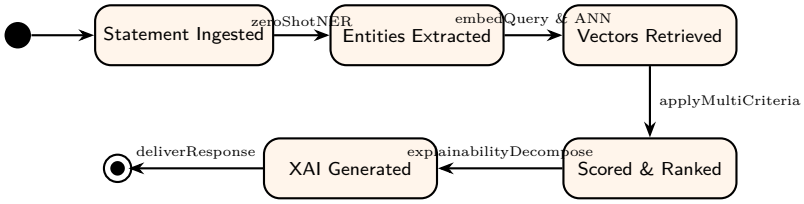


Figure 5.13: State Diagram for AI Matchmaking Inference Pipeline

5.3.6 State Diagram-06: Credential Attestation & Endorsement Status

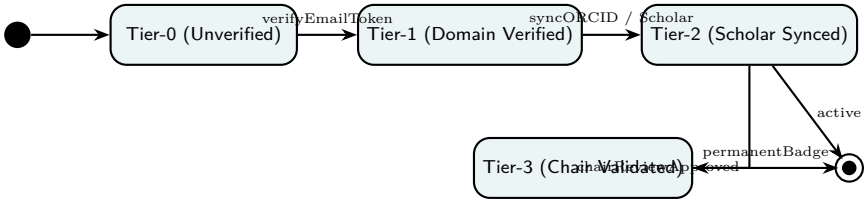


Figure 5.14: State Diagram for Credential Attestation Status

5.4 Entity Relationship Diagram

The relational data model supporting user management, taxonomies, capstone projects, matchmaking queries, advisory state machines, and audit logging is modeled in Figure 5.15.

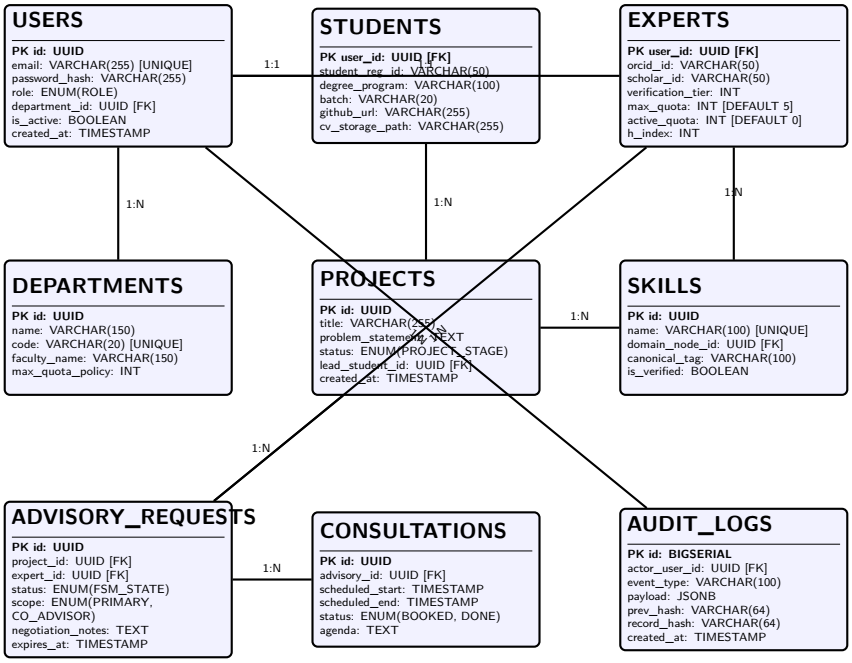


Figure 5.15: Entity Relationship Diagram (ERD) for UniExpert Data Schema

5.5 Class Diagram

The object-oriented software design, interface boundaries, and service abstractions are structured in the UML Class Diagram (Figure 5.16).

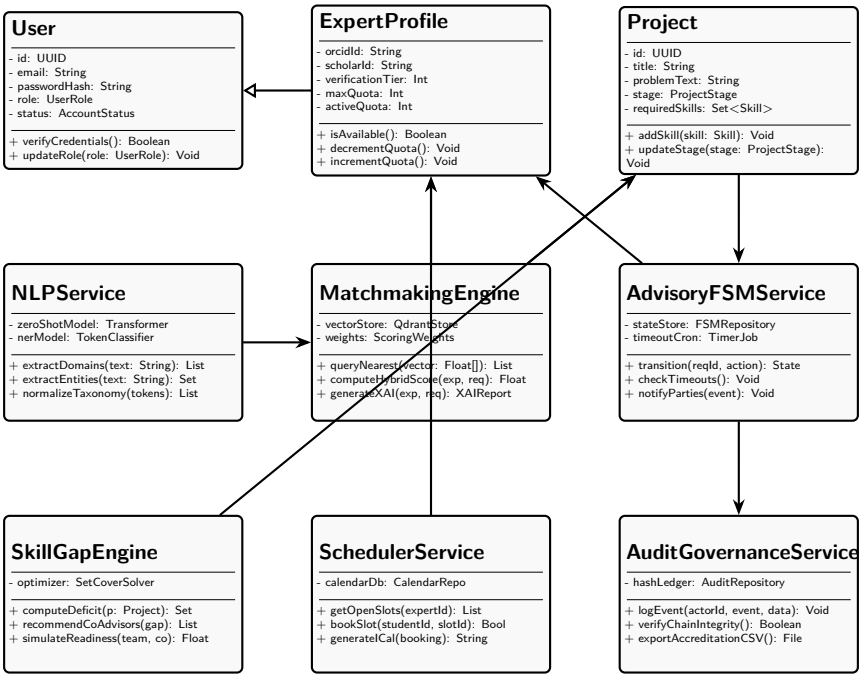


Figure 5.16: UML Class Diagram for UniExpert Architecture